Automation in Construction 188 (2026) 107041

Contents lists available at ScienceDirect

Automation in Construction

journal homepage: www.elsevier.com/locate/autcon

Lightweight graph retrieval-augmented generation agent for safety monitoring digital twins in small- and medium-scale hydropower projects

Chenfei Shao [a,b,c], Xiangnan Qin [d], Xing Li [e], Meng Yang [e], Dongyang Yuan [f], Sen Zheng [a,b,c,*]

[a] College of Water Conservancy and Hydropower Engineering, Hohai University, Nanjing 210098, China
[b] The National Key Laboratory of Water Disaster Prevention, Hohai University, Nanjing 210098, China
[c] National Engineering Research Center of Water Resources Efficient Utilization and Engineering Safety, Hohai University, Nanjing 210098, China
[d] School of Water Conservancy and Transportation, Zhengzhou University, Zhengzhou 450001, China
[e] Nanjing Hydraulic Research Institute, Nanjing 210029, China
[f] School of Infrastructure Engineering, Nanchang University, Nanchang 330031, China

ARTICLE INFO

Keywords:
Hydropower project
Safety monitoring
Digital twin system
Lightweight agent assistant
GraphRAG
ASR correction
KG maintenance

ABSTRACT

Intelligent agent assistants are becoming essential for efficient interaction in safety monitoring digital twin (DT) systems. However, their deployment in small- and medium-scale hydropower projects remains limited by complex modules, inefficient interaction, and constrained computing resources. To bridge this gap, this paper proposes a lightweight agent assistant for DT systems that integrates graph retrieval-augmented generation (GraphRAG)-enabled task-oriented context construction with Gemma-based large language model (LLM) response generation. Graph-constrained automatic speech recognition (ASR) correction improves intent recognition and hands-free operation. Additionally, a conflict resolution and dynamic update strategy supports lightweight knowledge graph (KG) maintenance. Ultimately, field validation in a practical medium-scale hydropower project DT system demonstrates that the agent assistant reduces users' operational cognitive load and ensures response accuracy. As a result, this paper provides a lightweight, low-cost, and portable DT interaction solution for safety monitoring and intelligent management in resource-constrained hydropower projects, while supporting human-machine collaboration in DT systems.

1. Introduction

In recent years, development of hydropower project safety monitoring DT systems has made significant advancements in terms of high-fidelity visual rendering and fusion of multi-source data [1]. A hydropower project is defined as an integrated system for water retention, water conveyance, power generation, and discharge regulation. It mainly consists of the reservoir, dam, spillway, intake structure, water conveyance structures, powerhouse, tailrace, slopes, and associated monitoring facilities [2]. The schematic of a hydropower project is illustrated as Fig. 1. Safety monitoring in hydropower projects is an undeniable necessity for the timely detection of abnormal structural characteristics, slope instability, or equipment malfunction. These problems may threaten structural safety, downstream communities, power-generation reliability, and the surrounding environment [3].

On the basis of the general concept of DT as virtual representations of physical systems supported by data connection, model updating, and decision-support functions [4,5], in this study, a hydropower project safety monitoring DT system is defined as a data-model-knowledge-integrated cyber-physical system that links the physical hydropower-station project with its virtual counterpart. It supports real-time state representation, monitoring analysis, safety assessment, early warning, and operational decision-making [6]. The development of these systems is critical to guarantee the high-efficiency and reliable operation of hydropower projects. Various functions of the system have been developed for a broad range of applications, such as safety monitoring information management, 2D and 3D integrated visualization, comprehensive monitoring analysis, safety assessment, early warning management, and so on [7,8]. According to recent studies, the core functions of hydropower-station safety monitoring DT systems are summarized in Fig. 2.

As illustrated in Fig. 2, modern safety monitoring DT systems of

* Corresponding author at: College of Water Conservancy and Hydropower Engineering, Hohai University, Nanjing 210098, China.
E-mail addresses: shao_chen_fei@hhu.edu.cn (C. Shao), qin_xn2015@zzu.edu.cn (X. Qin), xingli@nhri.cn (X. Li), ymym_059@126.com (M. Yang), dyyuan@ncu.edu.cn (D. Yuan), zheng_sen@hhu.edu.cn (S. Zheng).

https://doi.org/10.1016/j.autcon.2026.107041
Received 22 December 2025; Received in revised form 13 May 2026; Accepted 15 May 2026
Available online 18 May 2026
0926-5805/© 2026 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

hydropower projects primarily encompass seven functional modules. (1) Multi-dimensional perception & acquisition module serves as the foundation of the system, responsible for automatic data acquisition, digital visual inspection, multi-source heterogeneous data fusion [9]. (2) Data management & quality control module acts as the system's "filter" to ensure data availability and reliability [10,11]. (3) Visualization & interaction module provides the system with visualization information and interactive way, highlighting the DT capabilities [12,13]. (4) Analysis & modeling module is responsible for characteristic interpretation based on calculation and computation [14,15]. (5) Evaluation & diagnosis module is utilized in determining the health status of dam [16]. (6) Warning & response module supports for risk warning and closed-loop warning management [17]. (7) Operations & maintenance management module is adopted in full lifecycle management of hydropower project with the aim of safety operation [18,19]. Above all, the multi-dimensional and tightly coupled functional modules exhibit significant domain specificity and complexity.

As a result, the complexity of the system functional architecture has limited system adoption in daily applications [20,21]. Specifically, a significant mismatch exists between current interaction paradigms and increasingly intricate system functionalities, thereby constraining application efficiency. Traditional graphical user interfaces (GUIs) mainly rely upon the WIMP (Windows, Icons, Menus, Pointer) metaphor, which imposes a high cognitive load and requires extensive operational proficiency [22]. Accordingly, the primary challenges of the system function application lies in inefficient information retrieval and navigation bottlenecks. Traditional interaction modes impose high cognitive loads and steep learning curves, while lacking intuitive, natural language-based control capabilities, making it difficult to meet on-site demands for immediate response. The inadequacy of intelligent Human-Computer Interaction (HCI) remains a significant bottleneck in the development of digital twin fidelity and the evolution of the system maturity level [23,24]. Therefore, constructing an agent assistant equipped with intelligent human-computer interaction capabilities and rapid response mechanisms holds scientific significance for overcoming current application bottlenecks of safety monitoring DT system. In this study, an agent assistant refers to an execution-capable intelligent interface embedded in the DT system, which supports three core functions: instruction execution, information response, and information retrieval and visualization. Specifically, it is designed to understand user intent, retrieve domain knowledge, invoke valid system functions, and present relevant information within the DT system [25,26].

To date, in other research domains, the evolution of agent-based technology has provided a novel paradigm for achieving intelligent interaction and substantial progress has been made. Meng [27] and Motlaghzadeh et al. [28] have utilized game-theoretic approaches to model stakeholder interactions in ecological governance and climate adaptation. Bahrami et al. [29] and Aydin & Keleş [30] integrated evolutionary algorithms to optimize resource allocation and supply scheduling under uncertainty. However, despite their efficacy in specific scenarios, these traditional multi-agent systems (MAS) heavily rely on predefined rules and fixed strategic frameworks. This reliance on deterministic logic limits their practical applicability in complex system operation. In an effort to transcend the rigidity of conventional rule-based approaches, the convergence of LLMs and Retrieval-Augmented Generation (RAG) has become a hot issue in enabling intelligent system interaction [31,32]. Specifically, this paradigm leverages LLMs to provide agents with robust semantic comprehension and dynamic reasoning, and combines with RAG to ensure the factual consistency and operational accuracy of the system. For example, question answering (QA) systems, serving as the cornerstone of intelligent human-machine interaction, have undergone rapid technological development. The integration of LLMs and RAG has been widely utilized in optimizing interaction efficiency [33,34], demonstrating early efficacy in sectors like e-commerce [35] and healthcare [36,37]. Notably, Álvaro & Barreda [38] exploited RAG to reduce computational costs. Uhm et al. [39] developed a high-performance RAG-GPT model to generate accurate construction safety information. To surpass the limitation, Graph-Augmented Retrieval (GraphRAG) has emerged as a promising approach for enhancing the robustness and precision of interactive systems [40,41]. Unlike traditional relational databases, knowledge graphs (KGs) explicitly represent concepts, entities, and their relationships [42]. This structure provides richer semantic representation and greater flexibility in knowledge organization and expression [43,44]. By linking heterogeneous information through semantic relations, KGs can also improve information retrieval and support more context-aware knowledge access [45,46]. For instance, Yan et al. [47] constructed a KG to capture domain knowledge regarding the state prediction of hydraulic systems. Effectively integrating the structured knowledge into the architecture of intelligent agent assistant to enhance system interactive capabilities remains a critical research frontier. In view of this, Zhang et al. [48] have leveraged GraphRAG with LLMs to establish the QA system for engineering safety management. Similarly, Yang et al. [49] designed a multi-agent system based on the GraphRAG framework, successfully facilitating project operation and management decision-making. Miao et al. [50] have proposed a GraphRAG-based fault diagnosis method for high-speed train bogies based on GraphRAG. The validation results demonstrated that the method successfully balances high accuracy with strong interpretability.

In summary, significant strides have been made in developing agent assistants and interactive Q&A systems on the basis of LLMs and GraphRAG [51,52]. However, a critical gap still remains in their practical deployment for hydropower project safety monitoring. Existing GraphRAG-based agent systems are typically resource-intensive and

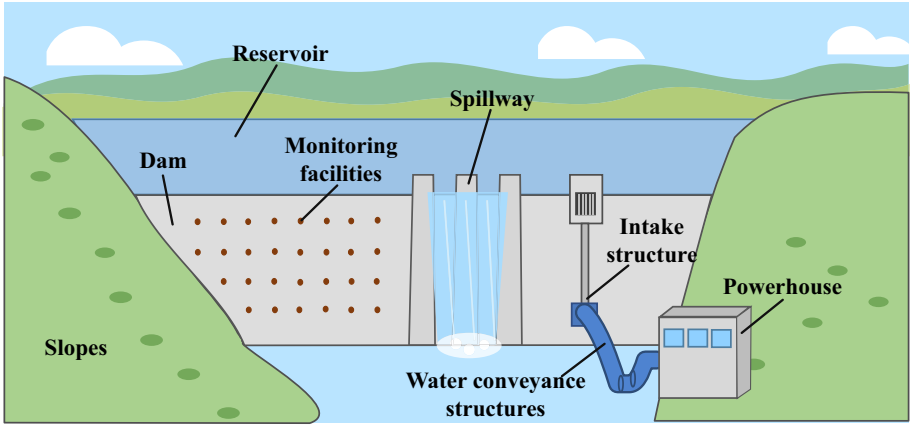


Fig. 1. Schematic of a hydropower project.

relatively static, making them unsuitable for small- and medium-scale projects that face strict constraints on computational hardware, domain expertise, and maintenance budgets. Merely applying general-purpose models is insufficient. Instead, a tailored and lightweight architecture is required to bridge the gap between advanced AI capabilities and the resource-constrained reality of field operation.To address this gap, this study is guided by the following central research question: How can a lightweight, execution-grounded, multimodal agent assistant be designed and integrated into a DT system for hydropower project safety monitoring to support reliable and efficient human-computer interaction in small- and medium-scale projects?

To answer this question, three specific sub-questions are further considered: (RQ1) How can domain knowledge about executable DT system functions, domain entities, and operational logic be formally organized to support reliable grounding of agent reasoning and action execution? (RQ2) How can GraphRAG and multimodal interaction mechanisms be combined within a lightweight framework to improve command understanding, professional question answering, and robustness to noisy speech input? (RQ3) To what extent can the proposed assistant improve response efficiency, retrieval accuracy, and operational usability when deployed in a real safety monitoring DT system?

To address these questions, this paper proposes a lightweight, multimodal agent assistant specifically designed for DT-based safety monitoring. The specific contributions are as follows:

(1) By leveraging the Gemma LLM and incorporating domain-specific operational logic and professional knowledge, a system-function-oriented knowledge graph (SFOKG) is constructed to explicitly represent executable system functions, domain entities, and operational logic. A conflict-resolution-based incremental update strategy is further proposed to support lightweight dynamic maintenance of the KG, avoiding full graph reconstruction and service interruption during field deployment. This design is particularly suitable for small- and medium-scale hydropower projects with limited computing resources, maintenance budgets, and technical support.
(2) On the basis of SFOKG, a lightweight GraphRAG-enabled agent assistant is developed. This framework integrates a perception-cognition-execution loop, enabling the agent assistant to reliably translate user intents into valid system commands and provide precise professional Q&A services under resource-constrained conditions of small- and medium-scale projects. Meanwhile, the proposed framework effectively mitigates hallucinations via KG-grounded action constraints.
(3) With the aim of enhancing the multimodal perception and execution accuracy of the agent assistant, a SFOKG-constrained ASR error correction method is proposed. This method maps noisy speech inputs to valid system-function nodes and domain concepts in the SFOKG, thereby preventing invalid command triggering. (4) The proposed agent assistant is deployed in a real DT safety monitoring system, demonstrating improved response efficiency and retrieval accuracy, as well as reduced user burden in complex workflows. As a result, a new paradigm of efficient human-machine synergy in DT system is provided. The results further demonstrate that the proposed method has strong practical potential for lightweight deployment and application in small- and medium-scale hydropower projects.

The remainder of this paper is organized as follows: Section 2 introduces the overall architecture and core functionalities of the hydropower project safety monitoring DT system and analyzes the functional requirements that motivate the research problem addressed in this study. Section 3 presents the construction of the lightweight, execution-grounded, multimodal agent assistant with GraphRAG-guided knowledge grounding developed to address these requirements. Section 4 evaluates the proposed approach through deployment in a practical medium-scale hydropower project, with emphasis on its effectiveness in improving response efficiency, retrieval accuracy, and intelligent human-computer interaction. Section 5 summarizes the main conclusions and outlines future research directions.

2. Overview of the developed safety monitoring DT system and functional requirements of the agent assistant

This section introduces the system context and integration strategy of the proposed agent assistant in the safety monitoring DT system. It first introduces the layered architecture and functional modules of the DT system, followed by an analysis of the deployment constraints and

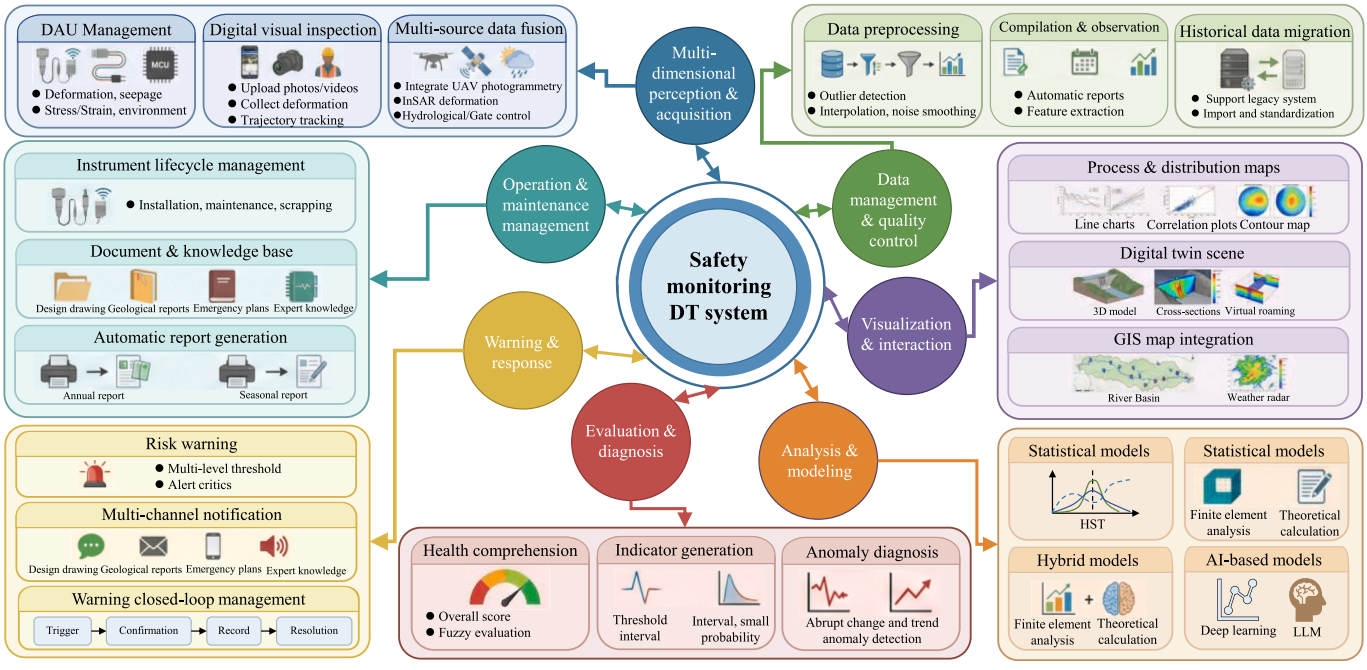


Fig. 2. Core functions of safety monitoring DT system.

interaction requirements of small- and medium-scale hydropower projects. Finally, the core functional requirements and integration architecture of the agent assistant are defined and presented.

2.1. Overall system architecture and functionality

In terms of technical architecture, the digital twin-based hydropower project safety monitoring system can be organized into five hierarchical layers: the information infrastructure layer, the data resource layer, the system service layer, the API interface layer, and the application presentation layer. The layered architecture effectively enhances system development efficiency and supports scalability and maintainability. The overall technical framework of the system is illustrated in Fig. 3.

As illustrated in Fig. 3, the architecture follows a bottom-up support logic and a top-down service delivery logic. Specifically, the DT-based hydropower project safety monitoring system is organized into five hierarchical layers, namely the information infrastructure layer, the data resource layer, the system service layer, the API interface layer, and the application presentation layer. This layered design clarifies the functional division of the system and improves its scalability, modularity, and maintainability. At the bottom, the information infrastructure layer provides the physical foundation for system operation, including monitoring and perception equipment, communication network equipment, and computing and storage equipment. Built upon this foundation, the data resource layer provides the underlying data and resource support required for upper-layer services and applications.

The system service layer provides the service support for various business applications within the system, enabling resource reuse across modules. This layer is composed of the data service layer, the foundational service layer, and the business service layer.

(1) Data service layer

This layer encompasses data reading and writing, stored procedures, data aggregation, file services, caching services, and data backup. By interfacing directly with the data resource layer, it supplies essential data support for both the foundational services and business services.

(2) Foundational service layer

This layer includes identity authentication, single sign-on, exception handling, and utility services, providing the core logical foundation required by the system's basic functions.

(3) Business service layer

This layer consists of domain-specific functional modules, including monitoring point management, data management, safety monitoring, numerical simulation, slope stability analysis, knowledge graph, risk analysis, rehearsal plan generation, comprehensive evaluation, and report generation. It also supports customized service deployment according to operational needs. The specific functionalities implemented by these modules are detailed in Table 1 and support further customized development.

Modern hydropower project safety monitoring systems always integrate specialized modules, resulting in low information retrieval efficiency and heightened error rates, especially during time-critical tasks that require fast response. Consequently, to overcome barriers caused by complex functional designs, the system needs to transition from a traditional imperative interaction paradigm (where the user dictates every step via clicks) to a declarative interaction paradigm. The proposed agent assistant serves as the essential human-system interface layer, allowing users to communicate their goals and intentions via natural language. This design will significantly enhance response speed, reduce execution errors, and promote a genuine state of human-machine synergy.

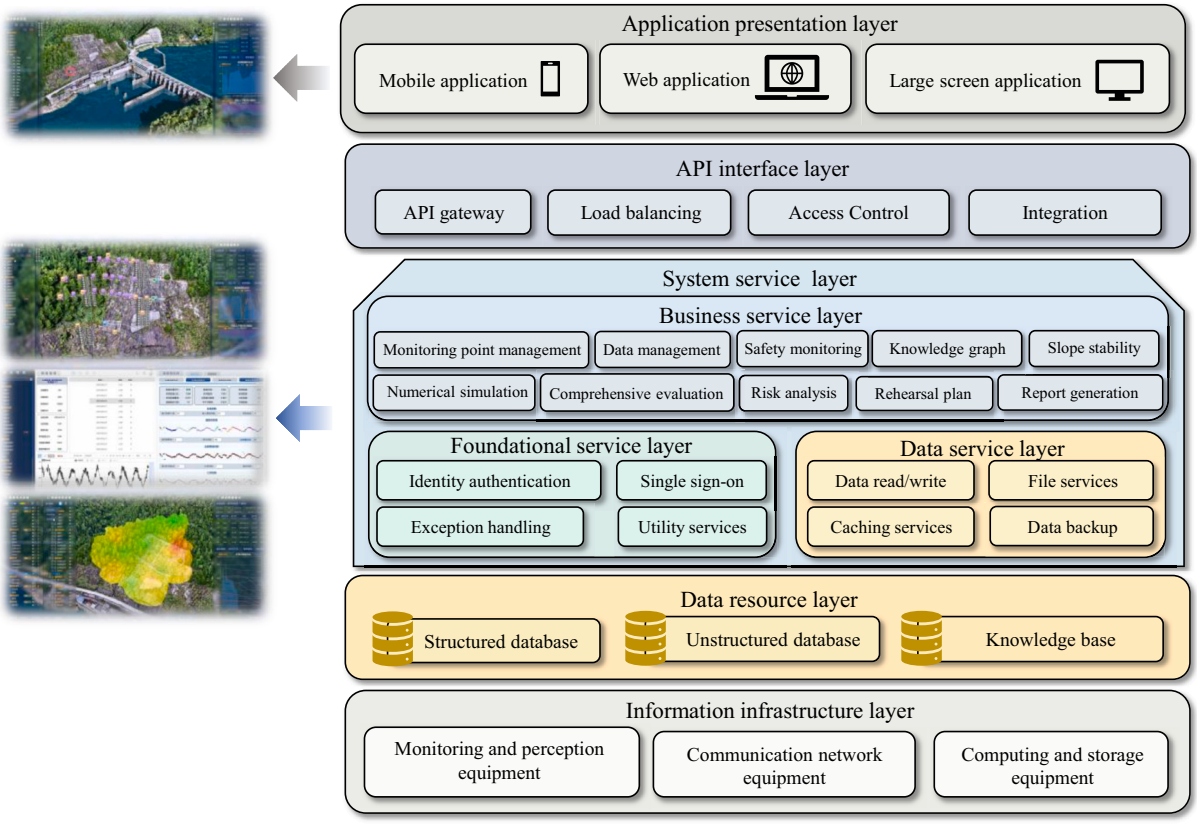


Fig. 3. Overall architecture of the hydropower project safety monitoring DT system.

Table 1
Specific functionalities implemented by the system modules.

Functional module	Specific functions
Monitoring point management	Manage the registration, encoding, positioning, and status of monitoring points, including the digital management of installation, calibration records, and maintenance history for monitoring equipments.
Data management	Responsible for the access, cleaning, storage, and integration of multi-source heterogeneous data (e.g., real-time, historical, environmental, model data), ensures unified data quality and consistency.
Safety monitoring	Conduct real-time acquisition, processing, and analysis of monitoring data, including threshold alarming, anomaly detection, trend forecasting, and automated early warning for exceedance data.
Numerical simulation	Perform simulation calculations for physical processes like structural stress, seepage field, and temperature field, based on engineering models and monitoring data, used for back analysis and forecasting structural responses under various conditions.
Slope stability analysis	Incorporate numerical analysis methods and calculate the factor of safety enabling quantitative assessment and analysis of slope stability.
Knowledge graph	Establish a dynamic domain knowledge base that supports both autonomous updates and manual modifications, allowing for the querying and expansion of the domain ontology.
Risk analysis	Integrate systematic risk assessment methodologies to conduct a multi-dimensional evaluation and quantitative analysis of engineering risks.
Rehearsal plan generation	Simulate and analyze the entire evolution process of engineering hazards, automatically generate tailored emergency response plans, thereby significantly enhancing engineering emergency response efficiency and decision-making capabilities.
Comprehensive evaluation	Integrate safety monitoring results, numerical simulations, and regulatory standards to perform quantitative and qualitative assessments of the overall safety status, generates safety level and risk evaluation reports.
Report generation	Automatically or semi-automatically generate periodic reports (daily, monthly, annual) and thematic reports, support data visualization and customized report output.

2.2. Scenario characteristics and design implications for small- and medium-scale hydropower projects

Although safety monitoring DT systems share a common technical foundation across hydropower projects, small- and medium-scale hydropower projects present a set of operational constraints that distinguish them from large flagship projects and directly affect the design of intelligent interaction mechanisms. In practice, such projects are often characterized by more limited computing resources, tighter maintenance budgets, and reduced availability of specialized personnel for long-term system operation and knowledge graph maintenance. At the same time, they still require timely access to monitoring information, engineering knowledge, warning interpretation, and function invocation in daily operation and emergency response. As a result, a practical tension exists between increasingly integrated DT system functionalities and the resource-constrained reality of field deployment.

In this scenario, the challenge lies not only in the complexity of the DT system itself, but also in the mismatch between complex functional modules and the operational capacity of end users. As shown in Table 1, the DT system integrates multiple domain-specific modules, including data management, safety monitoring, numerical simulation, slope stability analysis, knowledge graph management, risk analysis, rehearsal planning, comprehensive evaluation, and report generation. For users in small- and medium-scale projects, many routine tasks are highly goal-oriented and time-sensitive, such as querying the latest monitoring status, locating abnormal indicators, calling specific analytical functions, interpreting warning information, and switching to the relevant visualization view. Under conventional GUI-based interaction, these operations usually require multi-step navigation across different modules, which increases cognitive burden and raises the risk of operational errors, especially in scenarios requiring rapid response.

Therefore, the target scenario of this study calls for an agent assistant with several specific design requirements. First, the assistant should support lightweight deployment, so that it can operate under limited computing and maintenance conditions without relying on large-scale model infrastructures. Second, it should be execution-grounded, meaning that natural language interaction must be reliably mapped to valid system functions, domain entities, and operational logic rather than unconstrained text generation. Third, it should provide knowledge-grounded information support and function guidance, so that users can obtain accurate project-related answers and function guidance from structured domain knowledge rather than from general-purpose responses alone. Fourth, it should support robust multimodal interaction, especially in practical environments where speech input may be noisy and user commands may be incomplete or ambiguous. These requirements together explain why a lightweight, system-function-oriented agent assistant with GraphRAG-guided knowledge grounding is particularly needed in small- and medium-scale hydropower project safety monitoring DT systems.

2.3. Functional requirements of agent assistant

Based on the scenario characteristics and design implications discussed above, the core functionalities of the agent assistant for small- and medium-scale safety monitoring DT systems consist of three categories (as depicted in Fig. 4):

(1) Instruction execution

For explicit user operational commands (e.g., “Switch to a different project viewpoint”), the agent assistant possesses deterministic action mapping capabilities. By parsing action slots and object parameters within natural language inputs, it directly invokes underlying APIs to execute user instructions.

(2) Information response

For user queries about project specifications, monitoring status, or engineering interpretation (e.g., “What is the dead water level of the project?”), the agent assistant provides knowledge-grounded information support. It retrieves relevant context from the SFOKG and, when necessary, accesses the monitoring database to obtain grounded project-specific information, thereby generates coherent natural language responses.

(3) Information **r**etrieval and **v**isualization

Addressing user demands for querying real-time states or specific scenarios (e.g., “Show the current slope deformation cloud map”), the agent assistant integrates information retrieval, view navigation, and rendering capabilities. This enables automatic viewpoint transitions within the digital twin environment, dynamic chart loading, and scene rendering, realizing the joint visualization of monitoring data and 3D scenes.

The three task categories do not correspond to three isolated technical pipelines; instead, they share a unified GraphRAG-enabled cognition backbone. As depicted in Fig. 5, the proposed agent assistant adopts a front-end/back-end collaborative architecture. The back end of agent assistant consists of three layers, namely input processing, a shared GraphRAG-enabled cognition backbone, and downstream task branches, while the DT front end is responsible for interaction, execution, and rendering. Specifically, after a user request reaches the back end, the system first performs input processing, including ASR correction and intent classification, and then carries out graph-constrained entity

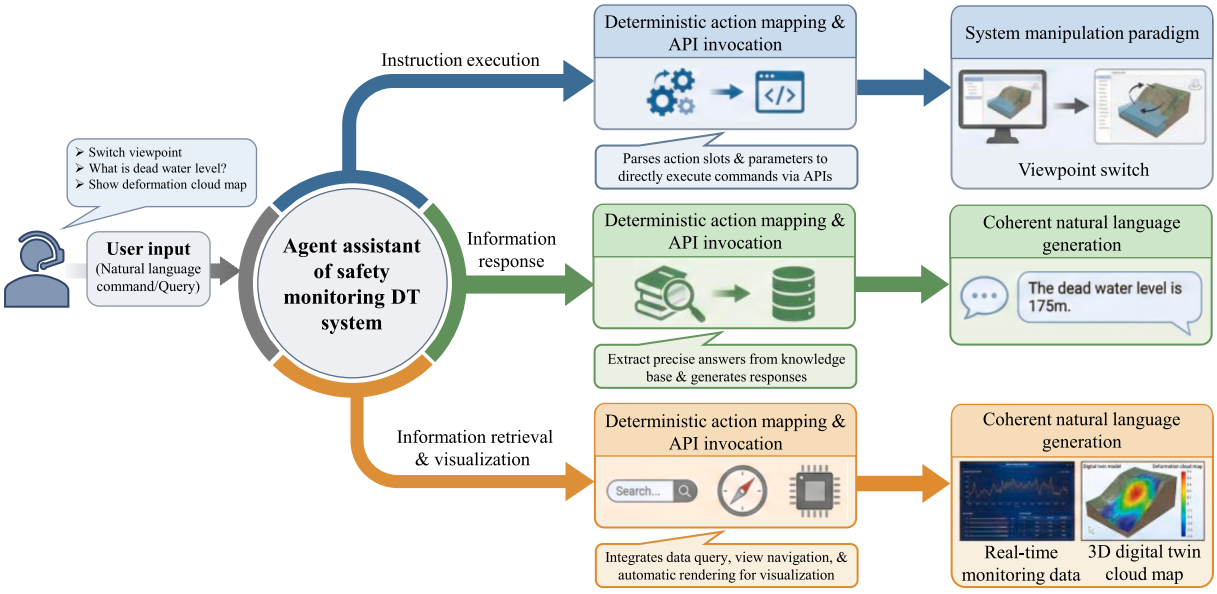


Fig. 4. Categories of agent assistant functional requirements.

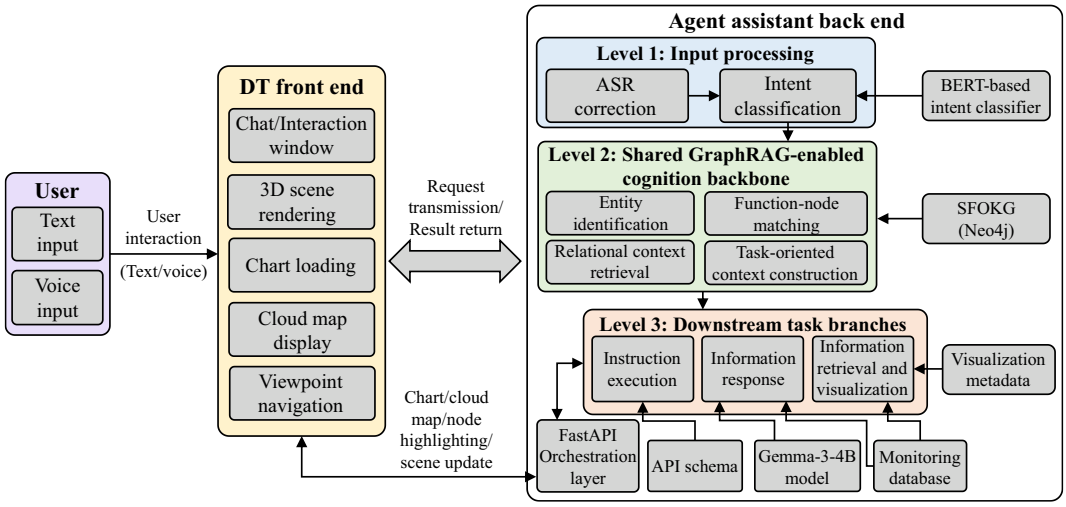


Fig. 5. Overall architecture of the agent assistant and its integration with the DT system.

identification, function-node matching, and relational context retrieval based on the SFOKG. On this basis, different downstream branches are activated according to the task type. For information response tasks, the GraphRAG-retrieved context is provided to the LLM model to generate a natural-language answer. When the query involves real-time or historical monitoring values, the monitoring database is further accessed to support grounded answer generation. For instruction execution tasks, after the relevant function nodes and invocation relations are identified, the retrieved context is transformed into a standardized command payload, and the command type, operation target, and parameters are returned to the DT front end for subsequent execution. For information retrieval and visualization tasks, the system further accesses monitoring databases and visualization metadata based on the GraphRAG output, organizes node IDs, chart requests, cloud-map parameters, or scene configuration instructions, and returns them to the DT front end for rendering, scene updating, and result presentation. In this way, the DT front end serves as the interaction, execution, and rendering terminal, whereas the agent assistant back end undertakes cognition, reasoning, retrieval, database access, and command generation.

3. Lightweight GraphRAG-enabled agent assistant construction method

The proposed lightweight agent assistant is designed as an intelligent interaction layer for the DT-based hydropower project safety monitoring system. As illustrated in Fig. 6, it aims to transform the traditional menu-dependent interaction mode of complex systems into a more efficient human-machine synergy mode. The agent assistant adopts a three-layer architecture, consisting of the perception layer, the cognition layer, and the execution layer, thereby forming an end-to-end closed loop from multimodal input understanding to knowledge-grounded reasoning and system action execution. It supports instruction execution, information response, and information retrieval and visualization within the safety monitoring DT system.

In the perception layer, speech and text inputs are processed through ASR, natural language pre-processing, and intent recognition. A graph-constrained ASR correction module is introduced to reduce ambiguity in spoken commands. In the cognition layer, the SFOKG serves as the shared knowledge backbone, while a conflict resolution and dynamic update strategy supports incremental graph optimization. Meanwhile,

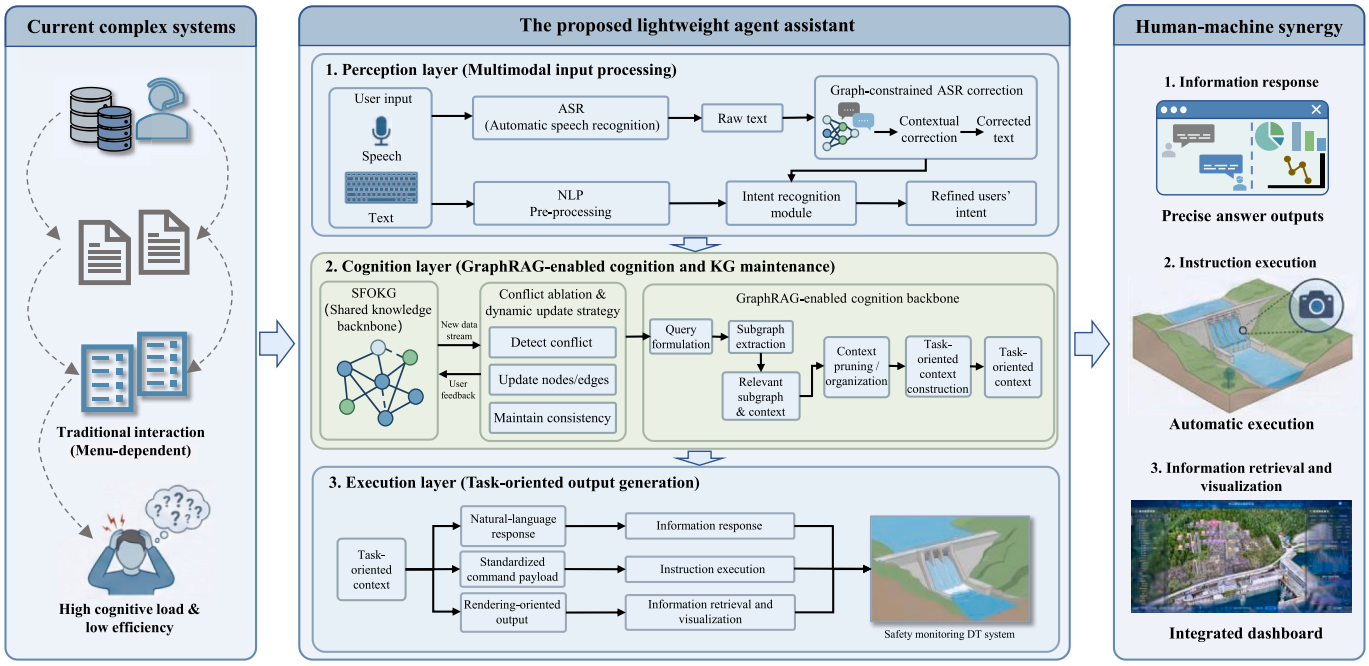


Fig. 6. Overall architecture of the proposed lightweight agent assistant.

GraphRAG enables knowledge-grounded retrieval and reasoning. In the execution layer, the retrieved context is transformed into different downstream outputs according to the task type, including natural-language responses for information response tasks, standardized command payloads for instruction execution tasks, and rendering-oriented outputs for information retrieval and visualization tasks. When necessary, the back end further accesses monitoring databases and visualization metadata to support grounded response generation and front-end rendering.

3.1. Construction of a multimodal perception layer with graph-constrained ASR correction method

The multimodal perception layer serves as the critical input channel for human-agent interaction, primarily dedicated to parsing user-provided text or voice commands. The core function involves extracting the user's intent and key operational instructions, integrating them with system state data, and relaying this context to the decision layer to trigger system operations. The system is engineered to support robust voice input, significantly enhancing the convenience and operational efficiency of human-computer interaction. Addressing the challenge of potential professional terminology misrecognition or semantic deviation introduced by the Automatic Speech Recognition (ASR) process, this study proposes a novel voice command correction method grounded in a domain knowledge graph constraint. Specifically, this method leverages the functional knowledge graph of the hydropower project safety monitoring system as a rigorous constraint to perform semantic matching and calibration on the ASR candidate results, thereby ensuring the accurate perception and high fidelity of user instructions.

Inspired by recent works in knowledge-augmented ASR, we implement a post-processing correction mechanism. While standard ASR models struggle with domain-specific terms, prior studies have demonstrated that incorporating phonetic and semantic information from an external knowledge base can significantly reduce error rates. Therefore, we define a hybrid scoring function based on phonetic similarity and semantic relevance.

ASR errors are predominantly characterized by acoustic ambiguities, typically manifesting as homophones or near-homophones. Consequently, direct character-level comparison between the ASR output and the knowledge graph entities is insufficient for accurate error detection. To address this phonological error pattern, we employ a Grapheme-to-Phoneme (G2P) conversion strategy. Specifically, G2P tools are utilized to transform both the ASR-recognized text T_{asr} and the candidate entities from the knowledge graph E_{graph} into standardized phoneme sequences. Crucially, these sequences are normalized to be tone-agnostic to maximize robustness against variations in speech prosody and accent. For example:

$$T_{asr} = \text{'洋压力'} \rightarrow [\text{'yang'}, \text{'ya'}, \text{'li'}] \tag{1}$$

$$E_{graph} = \text{'扬压力'}(\text{means uplift pressure}) \rightarrow [\text{'yang'}, \text{'ya'}, \text{'li'}] \tag{2}$$

The Levenshtein distance (LD) algorithm is employed to quantify the minimum number of single-character edits required to transform one phoneme sequence p_1 into the other p_2. This raw edit distance, denoted as $LD(p_1, p_2)$, is subsequently subjected to normalization, converting the distance metric into a similarity score ranging from 0 to 1, where a score of 1 indicates maximum phonetic similarity.

$$S_{phonetic}(p_1, p_2) = 1 - \frac{LD(p_1, p_2)}{\max(L(p_1), L(p_2))} \tag{3}$$

where $S_{phonetic}$ is the phonetic similarity score, $L(p_i)$ denotes the length of two phoneme sequences. Because the two phoneme sequences T_{asr} and E_{graph} are identical, $S_{phonetic} = 1$.

However, pure phonetic matching is susceptible to failure due to accented speech or ASR omissions. Therefore, semantic compensation is required, which is achieved by further comparing the vector embeddings of the ASR text and the knowledge graph entities. We leverage a pre-trained domain-specific Embedding model to map the two text segments into high-dimensional vectors:

$$V_{asr} = \text{Encoder}(T_{asr}) \tag{4}$$

$$V_{entity} = \text{Encoder}(E_{graph}) \tag{5}$$

Subsequently, the semantic proximity is quantified by computing the cosine of the angle between these two vectors, yielding the vector's Cosine similarity $S_{semantic}$:

$$S_{semantic}(T_{asr}, E_{graph}) = \cos(v_{asr}, v_{entity}) = \frac{v_{asr} \cdot v_{entity}}{\|v_{asr}\| \|v_{entity}\|} \quad (6)$$

This semantic evaluation serves as a robust mechanism for fault tolerance. For example, even if ASR incorrectly recognizes the phonetically different term "yang ya li" (洋压力) instead of the correct domain term "yang ya li" (扬压力), the strong co-occurrence of the shared term "pressure" (压力) ensures their embeddings remain close in the vector space, resulting in a high similarity score and successful correction.

Finally, a hybrid weighted approach is implemented to fuse the results from the two similarity metrics: the phonetic similarity score $S_{phonetic}$ and the semantic cosine similarity score $S_{semantic}$.The overall combined similarity score S_{com} is computed as a weighted sum:

$$S_{com} = \alpha S_{phonetic} + \beta S_{semantic} \quad (7)$$

In the context of ASR error correction, phonetic similarity typically holds greater discriminative power. Therefore, a higher weight is assigned to the phonetic component. This weighted combination allows the model to leverage both the high precision of phonetic matching and the robustness of semantic compensation for instruction correction. The final score S_{com} is utilized to make a definitive decision on entity correction and matching. A stringent threshold is applied to filter valid matches. When the calculated combined similarity score falls below this threshold, the system concludes that the knowledge graph contains no corresponding entity that is sufficiently aligned with the ASR output in terms of phonetic or semantic similarity. In such instances, the system is designed to reject the input and prompt the user for re-clarification of the instruction to ensure operational reliability.

The fusion weights α and β in Eq. (7), together with the acceptance threshold τ are not arbitrary. The above parameters and threshold are chosen to reflect the error mechanism of domain ASR and to balance correction effectiveness with operational safety. In hydraulic monitoring commands, ASR errors are predominantly driven by acoustic ambiguity, which makes phonetic similarity more discriminative than semantic similarity for correction. However, phonetic matching alone may degrade under accented speech or partial omissions; therefore, semantic similarity is incorporated as a fault-tolerant complement, leading to $\alpha > \beta$. The determination of parameters and thresholds for the graph-constrained ASR correction module is analyzed in detail in the case study of Section 4.

These parameters are tunable rather than fixed across projects. For new projects with different acoustic environments, dialects, or command vocabularies, (α,τ) can be re-calibrated using a small set of project-specific voice commands or via automated search.

3.2. Cognition layer construction for intent classification and high-level task planning

This section introduces the SFOKG construction and GraphRAG retrieval mechanism of the proposed agent assistant. It first describes how system-function knowledge is extracted, normalized, and stored as a graph. It then explains how the graph supports intent classification, local or global retrieval, and downstream task routing for response generation, instruction execution, and visualization.

3.2.1. Construction and clustering method of system function execution KG

To construct the system function execution KG serving the intelligent interactive agent assistant, this study utilizes LLM-based knowledge extraction aimed at efficiently transforming unstructured documentation (detailing system functions, operational logic, and implicit associations) into structured knowledge.

The key technical steps and aspects of this pipeline are detailed as follows:

(1) Semantic unit segmentation and operational entity identification: The corpus is first segmented into semantically complete text chunks to optimize LLM inference efficiency. Subsequently, we designed a domain-specific few-shot prompting strategy to guide the LLM. As illustrated in Table 2, the prompt consists of three components: (1) A role definition assigning the LLM as a "hydropower project expert". (2) A schema definition explicitly listing target entity types and relation types. (3) A set of few-shot examples covering typical edge cases. This structure ensures the LLM precisely identifies core entities such as functional modules and critical monitoring parameters while concurrently generating their normalized node attributes.

(2) Operational logic relation and dependency extraction: Building upon entity identification, the LLM further extracts structured semantic relations. We emphasize both the hierarchical "Contains" relations in the system structure and the logical correlations in function invocations. Accordingly, we extract functional dependencies, action-object associations, and operational sequences and represent them as triples (entity-relation-entity) to explicitly map the system call logic.

For instance, given the instruction "Enable the contour map in data display mode," the model not only extracts the entities but also constructs a hierarchical triplet: (Data Display, Contains, Show Contour Map). This structured output directly corresponds to the topological structure of the knowledge graph, ensuring that the agent accurately comprehends the subordination relationships and invocation paths of system functions. In parallel, for domain-specific queries, the model extracts semantic dependencies to support knowledge reasoning. Taking the aforementioned query regarding monitoring instruments as an example, the model constructs the relational triplet: (Slope Deformation, Monitored_By, Monitoring Instruments). Leveraging this topological structure, the system performs multi-hop retrieval within the KG to precisely locate specific device nodes such as "Inclinometer", "GNSS" and "machine vision". Consequently, it generates an accurate response stating, "Slope deformation monitoring instruments include: Inclinometer, GNSS and machine vision". This structured mapping mechanism ensures that the intelligent agent is capable of not only executing operational instructions but also conducting professional domain-

Table 2
Example of the few-shot prompt design for knowledge extraction.

Prompt component	Content/Template
Role definition	"You are an AI assistant specialized in hydropower project safety monitoring systems. Your task is to extract structured entities and relations from the provided technical documentation."
Schema definition	Target Entities: [View Mode], [System Function], [Monitoring Data], [Instrument Type], [Monitoring Category], [Located_In] Target Relations: [Contains], [Alias], [Triggers], [Related_To], [Monitored_By]
Few-shot examples	Input: "Switch to the dam view and enable the 3D Gaussian splatting effect." Output: {"Entities": [("Dam View", "View Mode"), ("Enable Gaussian Splatting", "System Function")], "Triples": [("Platform View", "Contains", "Dam View"), ("Settings", "Contains", "Enable Gaussian Splatting")]} Input: "Show the monitoring point value and the contour map for the reservoir." Output: {"Entities": [("Monitoring Point Value", "Monitoring Data"), ("Show Contour Map", "System Function")], "Triples": [("Show Monitoring Points", "Contains", "Monitoring Point Value"), ("Data Display", "Contains", "Show Contour Map")]} Input: "Please tell me what are the slope deformation monitoring instruments." Output: {"Entities": [("Slope Deformation", "Monitoring Category"), ("Monitoring Instruments", "Instrument Type")], "Triples": [("Slope Deformation", "Monitored_By", "Monitoring Instruments")]}
Constraints	"Output the result in valid JSON format. Map extracted terms to the standard node names defined in the KG"

specific Q&A based on graph knowledge.

(3) Entity normalization and graph quality assurance: To guarantee the accuracy and consistency of the KG, entity normalization is performed in both the initialization and maintenance stages. During seed graph construction, candidate entities and relations extracted by the LLM are reviewed against the predefined ontology, and ambiguous or semantically equivalent expressions are manually aligned to standard node names under expert supervision. This expert-involved curation is particularly important for executable function nodes, whose semantics must remain consistent with actual system functions and callable interfaces. After deployment, graph quality is further maintained through the automated conflict resolution and semantic-similarity-based deduplication strategy detailed in Section 3.4. In this way, the initialization-stage expert curation and the operation-stage lightweight maintenance mechanism jointly support the reliability of the SFOKG.

(4) KG construction and embedded storage: Finally, the extracted knowledge is organized into the SFOKG. The constructed SFOKG is instantiated and stored in the Neo4j graph database, which supports efficient retrieval and complex topological querying. Entities and their vector embeddings are saved, and a lightweight graph index is generated to fully support the subsequent GraphRAG application for command inference and function retrieval by the agent assistant. The KG construction process for the safety monitoring DT system with LLM-based knowledge extraction is illustrated in Fig. 7. On this basis, a custom Python-based GraphRAG pipeline is implemented to connect the SFOKG with the downstream reasoning and execution modules of the agent assistant. In the present implementation, Neo4j serves as the graph backend, while Cypher queries and the Neo4j Python Driver are employed for graph retrieval and neighborhood traversal.

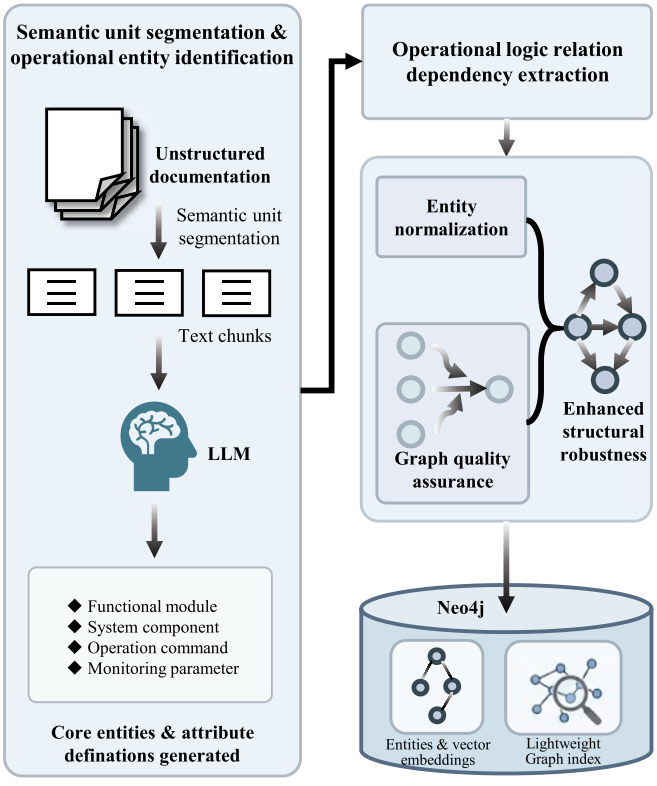


Fig. 7. Hydropower project safety monitoring system KG construction process using LLM-based knowledge extraction.

To enhance the semantic organization efficiency and local retrieval capability within the large-scale graph, GraphRAG integrates a hierarchical community partitioning mechanism during the graph optimization stage. The Leiden algorithm is applied to perform multi-level clustering on the entity-relation graph. This algorithm uses the structural connectivity strength among nodes to recursively optimize the partition. It ensures that entities within the same community are tightly connected, while inter-community connections remain relatively sparse. To accurately measure the partition quality and avoid the resolution limit problem faced by traditional methods, the Leiden algorithm utilizes the Constant Potts Model (CPM) as its objective function, which can be written as:

$$H = \sum_{c} \left[e_c - \gamma \binom{n_c}{2} \right] \tag{8}$$

where H represents the objective function of CPM, which is utilized to measure the quality of community partitioning, with a higher value indicating tighter intra-community connectivity, e_c denotes the actual number of edges within community c, n_c is the number of nodes in community, and γ is the resolution control parameter, $\binom{n_c}{2}$ represents the total number of possible edges within the community.

In our implementation, we set $\gamma = 1.0$ as the base resolution and limit the maximum community size to max_cluster_size $= 10$. Through this hierarchical approach, the knowledge graph is successfully partitioned into non-overlapping "semantic communities," providing an efficient structural foundation for subsequent localized retrieval and graph summarization tasks. After the community hierarchy is established, the system generates summaries for each level, serving as the graph's community-level semantic representations.

It is vital to note that this hierarchical clustering is a periodically executed optimization process. For high-frequency incremental updates (discussed in Section 3.4), the system maintains the existing community structure and only dynamically updates the attributes or local connections of affected nodes, thereby avoiding the computational overhead of real-time re-clustering.

3.2.2. *Knowledge graph retrieval and augmented generation*

To enable the robust inference of complex instructions and guarantee execution accuracy, the agent assistant employs a GraphRAG-enabled reasoning mechanism. In the present implementation, the GraphRAG module is realized as a custom Python-based lightweight pipeline. Specifically, the SFOKG is stored in Neo4j as the graph backend, while Cypher queries and the Neo4j Python Driver are employed for graph retrieval and neighborhood traversal. Semantic embeddings are generated using BGE-M3, and community partitioning together with graph summarization is implemented using the Leiden algorithm via the igraph and leidenalg libraries. The retrieved graph context is subsequently routed to different downstream modules according to the identified task type. For information-response tasks, it is provided to the Gemma-3-4B large language model, which is deployed with PyTorch and the Hugging Face Transformers framework, for knowledge-grounded answer generation. For instruction-execution tasks, it is transformed into a standardized command payload for schema-guided action mapping and DT system invocation. For information retrieval and visualization tasks, it is further combined with monitoring databases and visualization metadata to construct rendering-oriented outputs.

To support lightweight deployment, 4-bit quantization is adopted via bitsandbytes, and FastAPI is used to orchestrate the interaction among graph retrieval, LLM reasoning, and downstream execution modules. It should be noted that the SFOKG in this study was not initialized in a fully automatic manner. Instead, a human-in-the-loop construction strategy was adopted to ensure structural correctness and execution reliability in the safety monitoring scenario. Specifically, seed operational

instructions and function descriptions were first collected from system manuals, interface operation documents, help pages, and API specifications. These raw expressions were then manually normalized and rewritten into canonical command templates, and subsequently mapped to a predefined ontology schema. During this initialization stage, semantically equivalent expressions and functionally overlapping nodes were merged under expert supervision to improve graph consistency and avoid ambiguity in downstream retrieval and execution. Therefore, the SFOKG was not generated in a purely one-shot automatic manner; rather, its seed graph was carefully curated before being further expanded by the automated conflict resolution and incremental update strategy described in Section 3.4. The specific tools and their roles are described in Table 3.

The whole technical workflow is depicted in Fig. 8.

The agent assistant first subjects the input instruction to a BERT-based classifier implemented in the Transformers framework to determine the instruction's specific intent type, distinguishing between operational commands and informational inquiries. Based on this categorization, it dynamically triggers either a local or global search mechanism rooted in the GraphRAG paradigm and implemented in the custom Python-based retrieval pipeline.

(1) Local retrieval (for specific queries): For explicit, atomic instructions (e.g., 'Query monitoring point No. 3' or 'Switch to dam viewpoint'), a local retrieval is executed. In the current implementation, this process is realized through Cypher-based traversal over the target node and its 1-hop associated relations in Neo4j, so as to acquire specific attributes, parameter values, or function-level contextual information.
(2) Global retrieval (for macro queries): Conversely, for macro-level commands (e.g., 'analyze hydropower project safety status' or 'generate comprehensive report'), a global retrieval is initiated. This process aggregates pre-generated community summaries derived from Leiden-based graph partitioning, together with real-time incremental graph updates, to construct a comprehensive global context across multiple functional modules. The corresponding retrieval context is assembled in Python and then organized into a structured prompt for downstream reasoning.

The resulting retrieved graph context, serving as crucial task-oriented evidence, is subsequently routed to different downstream modules according to the identified intent. For information response tasks, it is provided to the LLM to generate a natural-language answer, with additional access to the monitoring database when grounded real-time or historical values are required. For instruction execution tasks, it is transformed into a standardized command payload for schema-guided action mapping and DT system invocation. For information retrieval and visualization tasks, it is further combined with monitoring databases and visualization metadata to construct rendering-oriented outputs, such as node IDs, chart requests, cloud-map parameters, or scene configuration instructions.

Table 3
Technical stack of the GraphRAG-enabled lightweight agent assistant.

Module	Tool / Framework	Role in the system
Programming environment	Python	Overall implementation and orchestration of the agent assistant
Large language model	Gemma-3-4B	Knowledge-grounded answer generation and reasoning support for information-response tasks
LLM inference framework	PyTorch + Hugging Face Transformers	Model loading and inference
Lightweight deployment	bitsandbytes (4-bit quantization)	Reducing memory footprint for lightweight deployment
Graph database	Neo4j	Storage and topological querying of the SFOKG
Graph access interface	Neo4j Python Driver + Cypher	Neighborhood traversal and graph retrieval
Intent classification	BERT-based classifier	Distinguishing different task types
Embedding model	BGE-M3	Semantic representation of graph entities and contexts
Community detection	igraph + leidenalg	Leiden-based graph partitioning and summarization
Backend service	FastAPI	Orchestration of retrieval, reasoning, and execution modules
Output format	JSON	Standardized command payload and rendering instructions

3.3. Execution layer for logic transformation and functional realization

The execution layer constitutes the final stage of the agent assistant and is responsible for translating the structured outputs of the cognition layer into three forms of downstream results: executable system commands, grounded natural-language responses, and rendering-oriented visualization instructions. To ensure precision and reliability in high-stakes hydropower project safety scenarios, this layer incorporates two core modules:

(1) Schema-guided action mapping (for operational tasks). Specifically for control-oriented intents, this module performs the deterministic translation of natural language intents into executable system commands via a retrieval-augmented approach:
1) API schema retrieval: the module queries the knowledge graph to retrieve the specific application programming interface (API) schema for the target function. This schema provides strict definitions for interface names, parameter data types, and constraint logic.
2) Context-aware parameter extraction: based on the retrieved function context and the API schema, the module extracts or completes the required arguments (e.g., temporal ranges, device IDs) through schema-guided parsing and constraint checking, ensuring alignment with the retrieved schema structure.
3) Standardized command generation: upon validation, the module synthesizes a standardized command payload in JSON format (e.g., {"action": "set_alarm", "target_id": "S01", "threshold": 50}), rendering it ready for system consumption.
(2) Closed-loop interactive feedback. This module completes the interaction cycle by converting digital commands or text outputs into observable system behaviors and user confirmations:
1) Multimodal response rendering: the digital twin front-end engine parses the JSON payload to drive specific interface operations, such as navigating to monitoring scenes, isolating data on charts, or initiating simulation workflows. For knowledge queries, it renders the generated expert response in the dialogue window or via voice synthesis.
2) Interaction closure: to ensure operational safety, the system provides immediate visual or haptic feedback (e.g., a confirmation dialogue for critical controls), thereby closing the loop from the initial voice input to the final functional execution or information delivery.

3.4. Lightweight KG maintenance method based on conflict resolution and incremental update strategy

To address the resource constraints of small- and medium-scale hydropower projects, this section introduces a lightweight KG maintenance method. By utilizing conflict resolution to eliminate topological redundancy and employing localized GAT embedding updates to bypass expensive global retraining, this mechanism effectively circumvents the dependency on high-performance computing clusters and enables agile deployment on standard hardware servers.

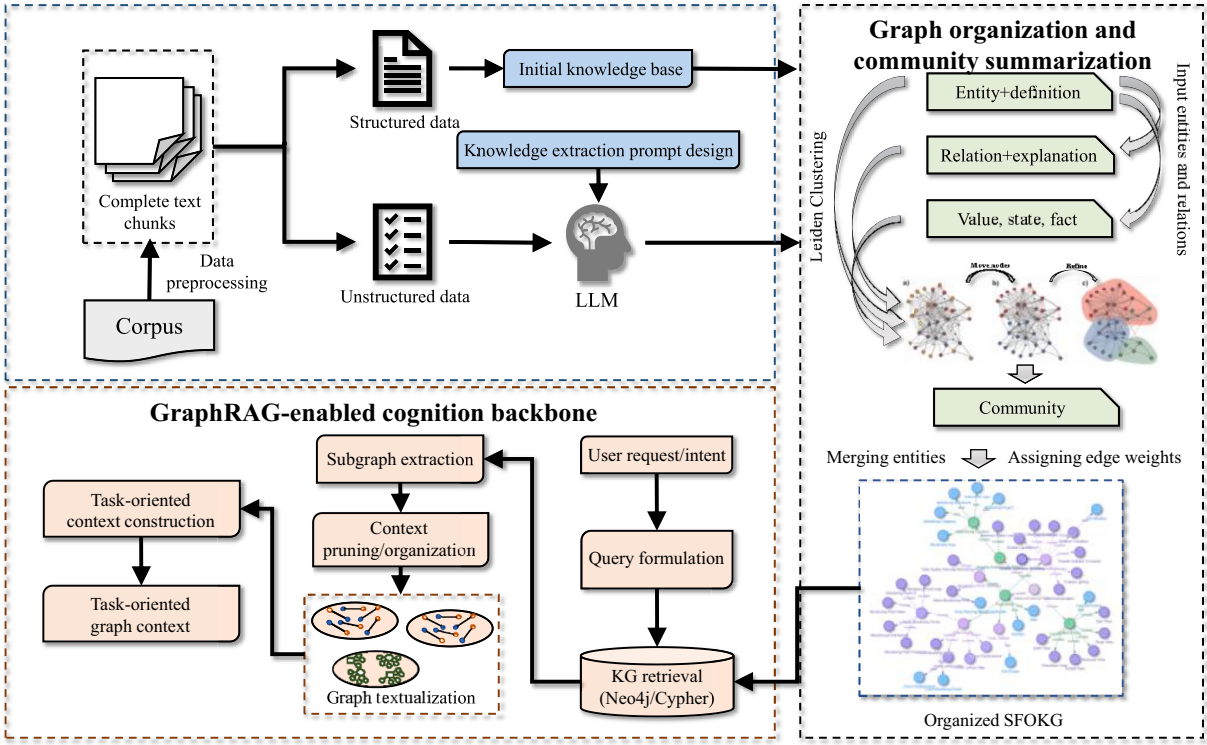


Fig. 8. Technical workflow of the shared GraphRAG-enabled cognition backbone.

3.4.1. *Mechanism of entity relationship deduplication and fusion*

As the system operates continuously, newly ingested descriptions (e.g., 'the rightmost gate') may result in entity redundancy or referential conflicts with pre-existing knowledge (e.g., 'Gate No. 1'). To address this, a conflict resolution strategy is introduced. By leveraging semantic similarity calculations to achieve entity alignment and normalization, this strategy facilitates the high-quality construction and dynamic scalability of the KG. The specific process steps of the conflict resolution strategy are detailed as follows:

(1) Entity alignment strategy

For any pair of entities e_i and e_j within the candidate entity set, their semantic vector embeddings, denoted as $h(e_i)$ and $h(e_j)$, are computed. Subsequently, the similarity metric $\mathrm{Sim}(h(e_i), h(e_j))$ is calculated. If the computed similarity meets or exceeds a predefined threshold τ (e.g.,$\tau = 0.85$), e_i and e_j are determined to be heterogeneous representations of the same underlying entity e_{ij}. Consequently, they are merged into a unified entity node e_{ij}. The merging operation preserves all attributes and adjacency relationships of the original entities, while the most frequently occurring mention is selected as the canonical name for the merged entity. This process is formalized in Eq. (9):

$$Sim\left(h(e_i), h(e_j)\right) \geq \tau \Rightarrow e_{ij} = merge(e_i, e_j) \tag{9}$$

where $merge(e_i, e_j)$ denotes the operation of coalescing nodes e_i and e_j into a new node e_{ij} that inherits their attributes. Following this step, the candidate entity set is updated to $V_{cand} := (V_{cand} \backslash e_i, e_j) \cup e_{ij}$, while the candidate relationship set R_{cand} retains the original associations.

(2) Relationship resolution strategy

Upon completion of entity merging, the relationship set is scrutinized for potential conflicts. In instances where multiple candidate relationships $\{r_1, r_2, \ldots\}$ exist between the same entity pair (s, o), a selection is made based on occurrence frequency. The relationship with the highest frequency is retained as the primary relationship, as expressed in Eq. (10):

$$r_{max} = \arg \max_{r \in \{r_1, r_2, \ldots\}} freq(r) \tag{10}$$

The remaining relationships are stored as synonymous labels. The updated relationship set R' replaces the original set containing conflicts. Finally, the definitive entity set V', relationship set R', and the corresponding KG are output. Crucially, this strategy performs entity-relation extraction solely on new document chunks, thereby avoiding the computational overhead associated with full graph reconstruction.

3.4.2. *KG dynamic maintenance mechanism*

To facilitate the continuous ingestion of new documentation and Q&A content by the system agent assistant, a lightweight incremental update strategy is introduced. This approach circumvents the significant computational overhead associated with full graph reconstruction. The strategy is designed to incrementally integrate newly extracted knowledge into the existing Knowledge Graph (KG). Let the current KG be denoted as:

$$G = (V, R, \mathscr{T}) \tag{11}$$

where V represents the set of entity nodes, R denotes the set of relation types, and $\mathscr{T} \subseteq V \times R \times V$ represents the set of triples. The input consists of a set of triples $\Delta\mathscr{T} = (s_i, r_i, o_i)$ extracted from new documents or QA pairs, and the output is the updated graph $G' = (V', R', \mathscr{T}')$ following the merger of new knowledge. As illustrated in Fig. 9, the proposed incremental optimization process mainly includes entity-relation extraction, conflict resolution for entity and relation resolution, dynamic graph structure updating, and GAT-based embedding refinement for newly added nodes. The core processing workflow is as follows:

New entity relations are structured into triples within $\Delta\mathscr{T}$. For each triple $(s, r, o) \in \Delta\mathscr{T}$, entity matching and merging are performed. For the subject s: If there exists an entity $v \in V$ in the graph such that the semantic similarity $Sim(h(s), h(v)) \geq \tau$, then s' is set to v, and their attributes are merged. Otherwise, s is added to V as a new node, with $s' =$

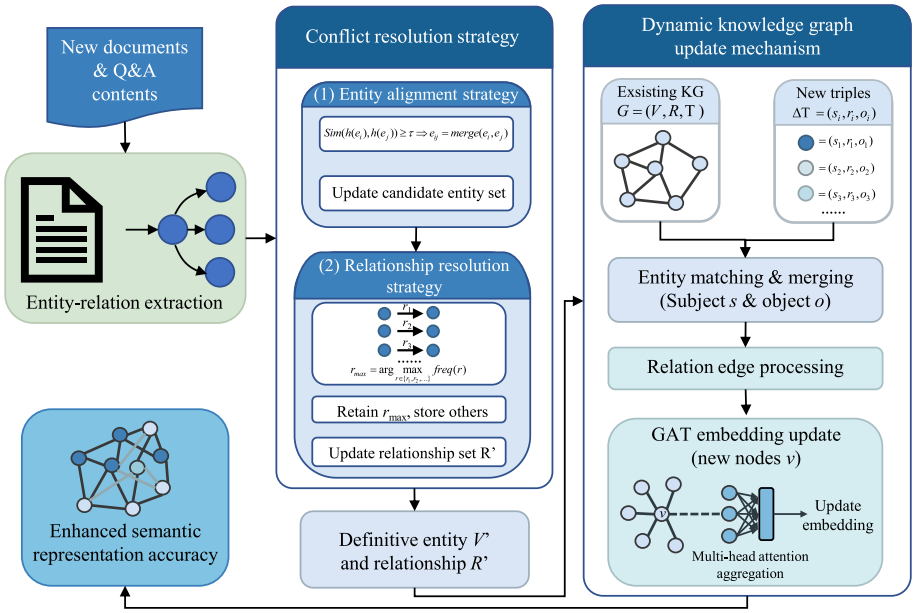


Fig. 9. Dynamic KG optimization process based on conflict resolution strategy and incremental update strategy.

s. For the object o: Similarly, if a corresponding entity $v' \in V$ exists that is similar to o, o' is set to v' and attributes are merged. Otherwise, o is added as a new node with $o' = o$. Subsequently, relationship edges are processed: if the triple (s', r, o') already exists within $\mathscr{T}$, its attributes (e.g., weight or confidence score) are updated. If it does not exist, (s', r, o') is inserted as a new edge into $\mathscr{T}'$, and the relation type set R is updated accordingly. Finally, Graph Attention Networks (GAT) are utilized to update the representation vectors of newly added nodes. For each new node v, neighbor information is aggregated via multi-head attention mechanisms to compute the updated embedding $h(v')$, thereby enhancing semantic representation accuracy. Consequently, through the design and implementation of conflict resolution and dynamic update strategies, the system agent assistant efficiently achieves incremental KG updates and optimization when ingesting new documents, significantly elevating the logical coherence and practical utility of the KG.

4. Case study

This section presents the case study and evaluation of the proposed agent assistant. It first introduces the medium-scale hydropower project used for system validation, and then presents the implementation results, including the lightweight LLM, SFOKG, and practical DT system interfaces. Finally, the assistant is evaluated in terms of response performance, interaction efficiency, lightweight deployment, resource consumption, and user satisfaction.

4.1. Project overview

A medium-scale hydropower project is located in the middle reaches of the Jinxi River, a tributary of the Minjiang River Basin. It lies 41 km upstream of Chitan and 25 km downstream of the urban area of Jiangle County. The project primarily comprises a concrete gravity dam, a retaining dam, a spillway, a powerhouse, and an indoor step-up substation. Situated at a site controlling a catchment area of 5388 km^2, the

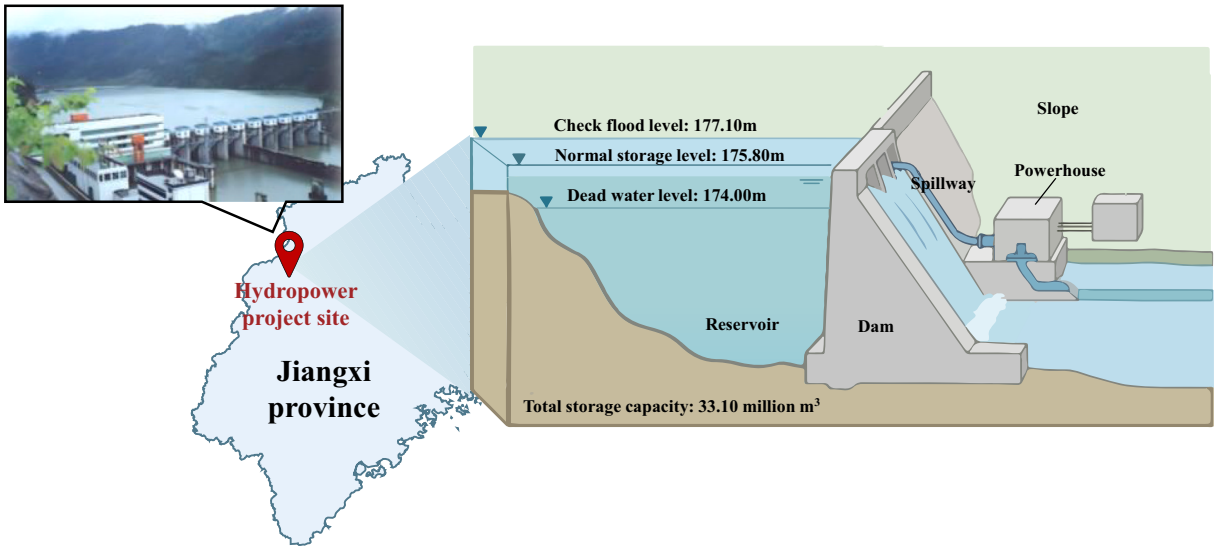


Fig. 10. Project location and schematic diagram of the hydropower project.

reservoir has a total storage capacity of 33.10 million m^3. Key hydrological parameters include a dead water level of 174.00 m, a normal storage level of 175.80 m, and a check flood level of 177.10 m. The station operates with an average hydraulic head of 14.0 m. Fig. 10 illustrates the project location and provides a site photograph.

4.2. Development of lightweight agent assistant based on GraphRAG

This section presents the implementation results of the proposed agent assistant, including the selection of the lightweight LLM, the constructed SFOKG, and the practical system interfaces. Representative application cases are further provided to demonstrate the agent assistant's capabilities in natural-language command execution, monitoring-information retrieval and visualization, and domain-oriented information response.

4.2.1. Comparative analysis and model selection

To ensure a lightweight and efficient design for the agent assistant, Gemma-3-4B was selected as the foundational LLM. Gemma-3-4B is a compact instruction-tuned model in the Gemma 3 family, officially released for resource-accessible deployment settings and supporting a 128 K context window, which makes it suitable for local or on-premise deployment in resource-constrained environments [53]. The primary rationale for this selection lies in its balance between compact model size, practical reasoning capability, and deployment feasibility, which is critical for on-premise deployment. This is particularly important for ensuring the security and data sovereignty of sensitive engineering safety monitoring data in small- and medium-scale hydropower projects. Gemma-3-4B provides sufficient reasoning capability for the intended DT interaction tasks, while remaining suitable for practical integration with the proposed GraphRAG and SFOKG based framework.

To rigorously justify this selection over other reasonable alternatives, Gemma-3-4B, Phi-3-mini, and Qwen3-4B were compared. These models are representative lightweight open models with comparable parameter scales, public model documentation and feasible configurations for local deployment. Phi-3-mini is a 3.8B instruct model available in lightweight deployment settings with 4 K and 128 K context variants, while Qwen3-4B is a 4.0B model with a larger context window [54,55]. These baselines were therefore selected to provide a fair engineering comparison among compact models in terms of reasoning capability, context support, deployment efficiency, and interoperability with the proposed DT-oriented agent framework.

As detailed in Table 4, the purpose of this comparison is not to claim that Gemma-3-4B is universally superior to all other compact models, but to identify a balanced engineering choice for our target scenario. In particular, Gemma-3-4B was finally adopted because it provides sufficient reasoning capability for the intended DT interaction tasks, while remaining suitable for stable on-premise deployment and practical integration with the proposed GraphRAG- and SFOKG-based framework.

On the basis of Table 4, Gemma-3-4B was selected mainly because of its advantages in practical deployment efficiency and architectural suitability for our specific application. The reasons for this selection are as follows:

(1) Although Gemma-3-4B has a slightly larger parameter count (4.3B) compared to Phi-3-mini (3.8B), it exhibits highly competitive effective memory utilization. Specifically, Gemma's large vocabulary size (256 k) allows for more efficient tokenization, encoding the same text length into fewer tokens. This efficiency offsets the computational overhead of the larger parameter size, resulting in lower end-to-end latency during inference. Furthermore, regarding context window management, unlike Qwen3 (262 K), which requires a significantly larger KV cache to fully utilize its context, often leading to out-of-memory (OOM) risks on consumer-grade hardware, Gemma-3 strikes a practical balance between processing capacity and memory constraints.

(2) Crucially, despite the gap in some benchmark scores, Gemma-3-4B achieves a BIG-Bench Hard (BBH) score of 72.2, which is highly competitive with the strong baseline of Phi-3-mini (73.5). This indicates that the model possesses sufficient logical reasoning capabilities to handle the tasks in hydropower project safety monitoring DT systems.

Therefore, we do not claim that Gemma-3-4B is universally superior to all alternatives. Rather, it was selected as a balanced engineering choice offering architectural agility, deployment stability, and sufficient reasoning capability for our resource-constrained system in small- and medium-scale hydropower projects.

4.2.2. Constructed SFOKG for hydropower project safety monitoring DT system

Fig. 11 indicates the core segment of the constructed KG, which mainly covers engineering monitoring, scene rendering, viewpoint navigation, and interface interaction. It can be seen that the graph utilizes green nodes (e.g., “Hydropower Station”, “Monitoring Location”, “Platform View”) as central hubs to construct the system framework. It clearly defines the various functional modules of the hydropower station, drilling down from macro-level “Monitoring Areas” to specific “Instrument Types” and “Monitoring Point IDs.” The left side of the graph features a dense cluster of entities related to engineering safety. The basic information covers monitoring locations, projects, and specific instrument details. Advanced analysis functions primarily include nodes such as “Monitoring Point Prediction”, “Alarm Monitoring Points”, and “Show Contour Map”. This demonstrates that the system is capable not only of querying historical data but also of performing monitoring data modeling and visual analysis. The right side of the graph illustrates the fine-grained control capabilities over the three-dimensional digital twin scene. Rendering technology control function specifically introduces nodes for enabling and disabling “Gaussian Splatting”, as well as controlling Ambient Occlusion, Lighting, and Weather conditions. This indicates the integration of cutting-edge real-time rendering technologies within the system. Through the “Camera Position” node, the graph links preset viewpoints such as “Dam View”, “Slope View”, and “Panoramic View”, supporting the agent assistant in rapidly switching scene perspectives based on user instructions.

The graph extensively employs “Alias” relationships (e.g., near “Instrument Type” or “Interface” nodes). These alias nodes play a critical role in the construction of the intelligent agent. They allow users to use non-standard terms or colloquial expressions during queries. The system can map these inputs to standard operational commands via the graph, thereby achieving highly robust natural language interaction.

While Fig. 11 visualizes the core segment of the SFOKG, Table 5 provides the comprehensive quantitative statistics of the entire SFOKG. As illustrated in the table, the graph integrates 1245 nodes and 3890 edges, forming a specific domain knowledge base. The limited number

Table 4
Comparative Analysis of Lightweight LLMs (<5B Parameters).

Model	Params	Vocab Size	Context	MMLU	BBH	IFEval	Evidence (URL)	License
Gemma-3-4B (Ours)	4.3B	256 k	128 K	59.6	72.2	55.4	Hugging Face: google/gemma-3-4b-it	Gemma Terms of Use
Phi-3-mini	3.8B	32 k	128 K	68.8	73.5	70.2	Hugging Face Model Card	MIT
Qwen3-4B	4.0B	152 k	262 K	84.2	~80+	83.4	Hugging Face Qwen3 Card	Apache 2.0

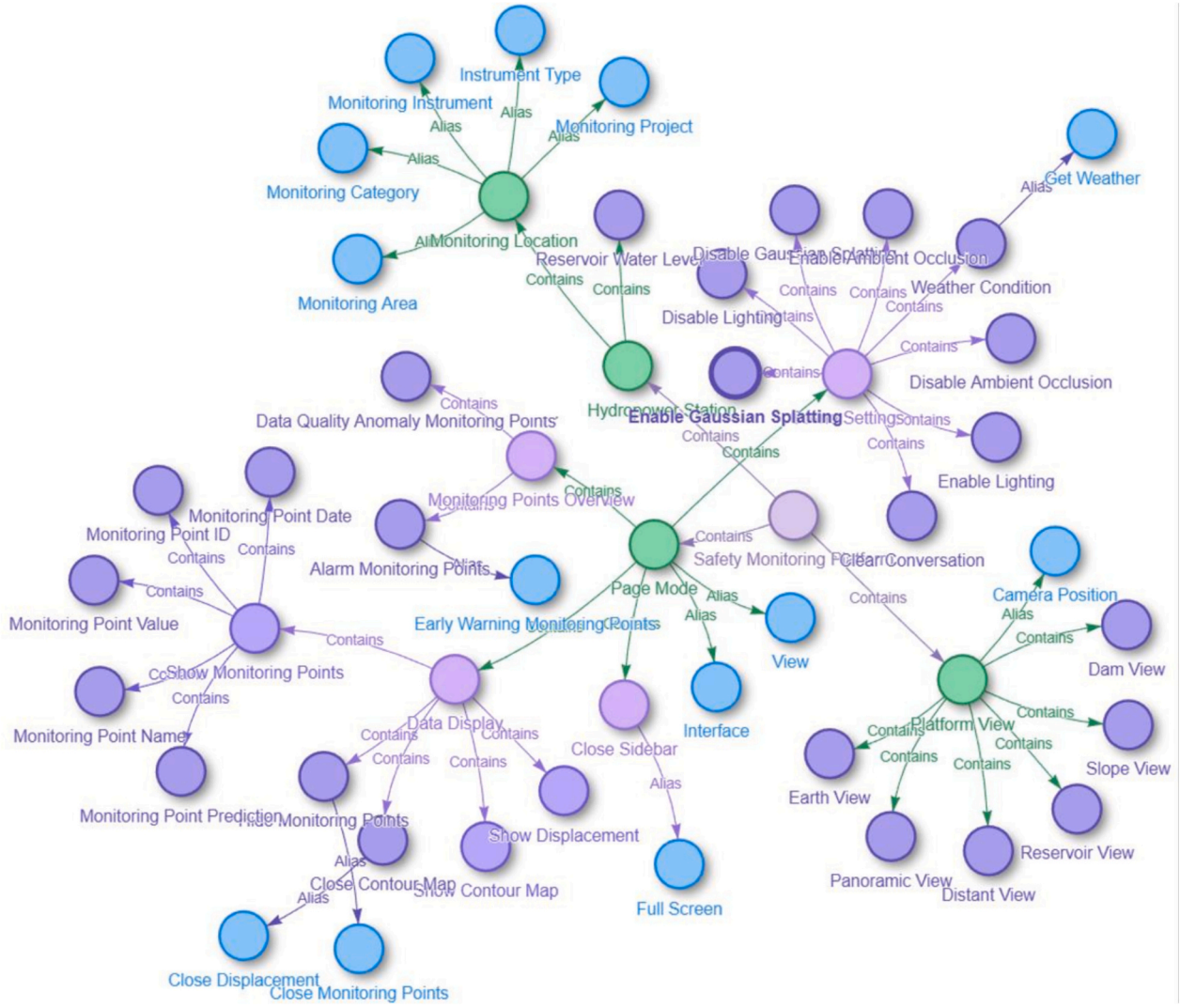


Fig. 11. Core segment of the SFOKG for the hydropower project safety monitoring digital twin system.

Table 5
Quantitative statistics of the constructed SFOKG.

Statistic category	Count/Description
Total entities (nodes)	1245
Total relations (edges)	3890
Entity types	5 (View Mode, System Function, Monitoring Data, Instrument Type, Monitoring Category)
Relation types	6 (Contains, Alias, Triggers, Related_To, Monitored_By, Located_In)
Average degree	3.12
Storage size	~1.5 MB (Lightweight)

of entity and relation types reflects the focused design of the schema, which avoids data redundancy and supports efficient inference.

4.2.3. Practical system interfaces of the agent assistant

The login interface of the digital twin system for safety monitoring of the medium-scale hydropower project is exhibited in Fig. 12. The system interface includes username and password input fields, as well as a system function selection button.

Fig. 13 presents the main operational interface of the proposed DT-based hydropower project safety monitoring system. As depicted in Fig. 13, the interface adopts an integrated dashboard layout in which the central area displays the three-dimensional DT scene of the hydropower project, while the surrounding panels provide coordinated access to key functional modules, including hydrological monitoring, environmental information, instrument status, data quality assessment, safety warning, risk monitoring, and comprehensive evaluation. This integrated interface visually reflects the information-intensive and highly coupled nature of hydropower project safety monitoring tasks.

More importantly, Fig. 13 demonstrates how multi-source monitoring information, model analysis results, and visualized DT representations are unified within a single interactive environment. The dialogue box located at the bottom center serves as the entry point for the proposed agent assistant, enabling users to issue commands and retrieve information through both text and voice inputs. Therefore, this figure not only shows the main interface of the system, but also highlights the practical necessity of introducing the lightweight agent assistant to reduce operational complexity and improve interaction efficiency in such a functionally dense DT system.



Fig. 12. Login interface of the system.



Fig. 13. Main interface of the proposed DT-based safety monitoring system.

As depicted in Fig. 14, the knowledge graph management module serves as the foundation for constructing and maintaining the knowledge base of the proposed lightweight agent assistant. This module comprises four fundamental functions: knowledge base management, natural language processing (NLP) extraction, triplet generation, and dynamic graph maintenance.

(1) Knowledge base management: This function enables users to autonomously upload files, facilitating the real-time construction and updating of the KG.
(2) NLP extraction: This function allows users to select large language models (LLMs) and configure specific parameters, including maximum token limit, extraction strategy, temperature, batch size, Top-P, Top-K, and repeated penalty, thereby enabling flexible knowledge extraction from heterogeneous texts.
(3) Triplet generation: This function converts extracted information into structured triples and supports parameter settings such as generation pattern, confidence threshold, maximum triplet number, entity deduplication, relationship verification, relationship filtering, and output format. A conflict resolution strategy is incorporated within this function to prevent the generation of duplicate entities.
(4) Knowledge graph management: Utilizing a dynamic maintenance mechanism, this function enables the dynamic management of the KG. It allows users to autonomously configure and modify key

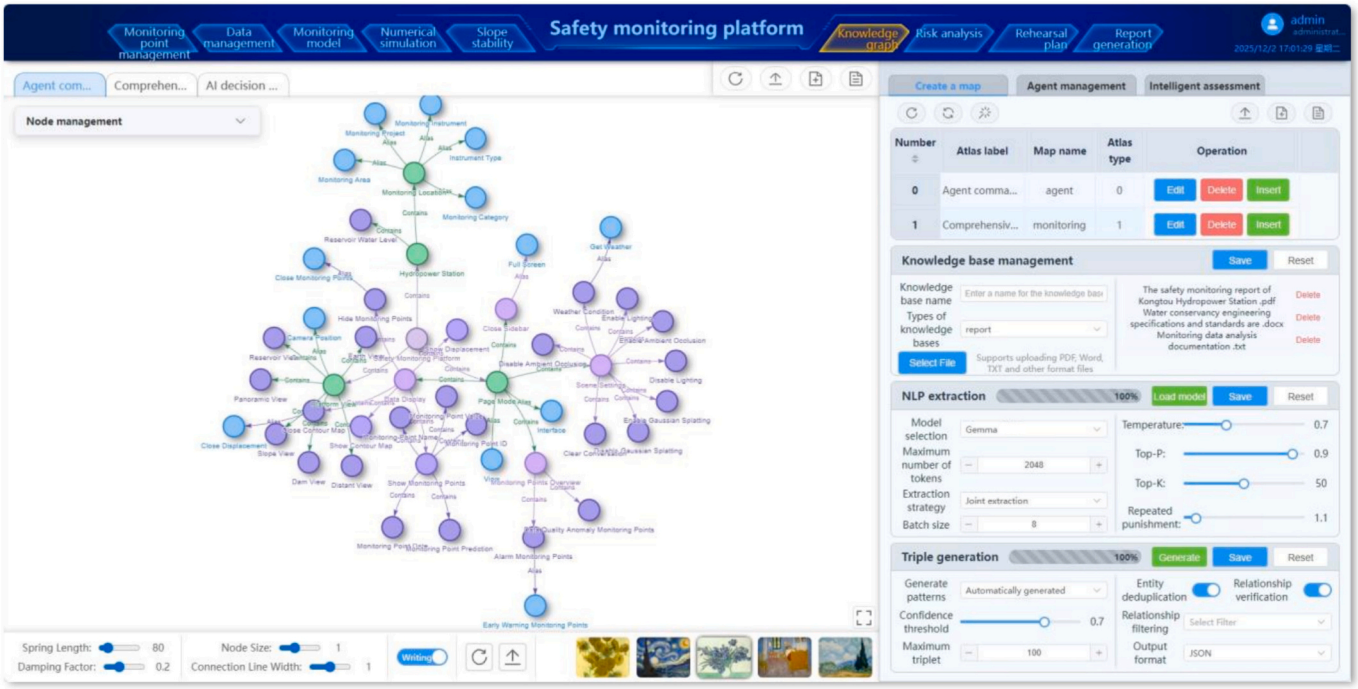


Fig. 14. Interface of knowledge graph management module.

metadata, thereby continuously refining the system functional KG through a domain knowledge maintenance mechanism.

Fig. 15 illustrates the agent assistant management module of the system. The functional module primarily comprises three key configuration areas: Large Language Model (LLM) settings, retrieval-enhanced construction settings, and graph retrieval settings.

(1) LLM settings: Users can customize model parameters, specifically including the system prompt, backbone model selection, maximum token number, temperature, Top-P, and Top-K.
(2) Retrieval-enhanced construction settings: Designed for the configuration of retrieval-augmented generation, this allows users to select knowledge bases, embedding models, mixed-search weights, retrieval quantity, context length, and similarity thresholds.
(3) Graph retrieval settings: This section enables flexible configuration of search strategies and relationship types, thereby precisely

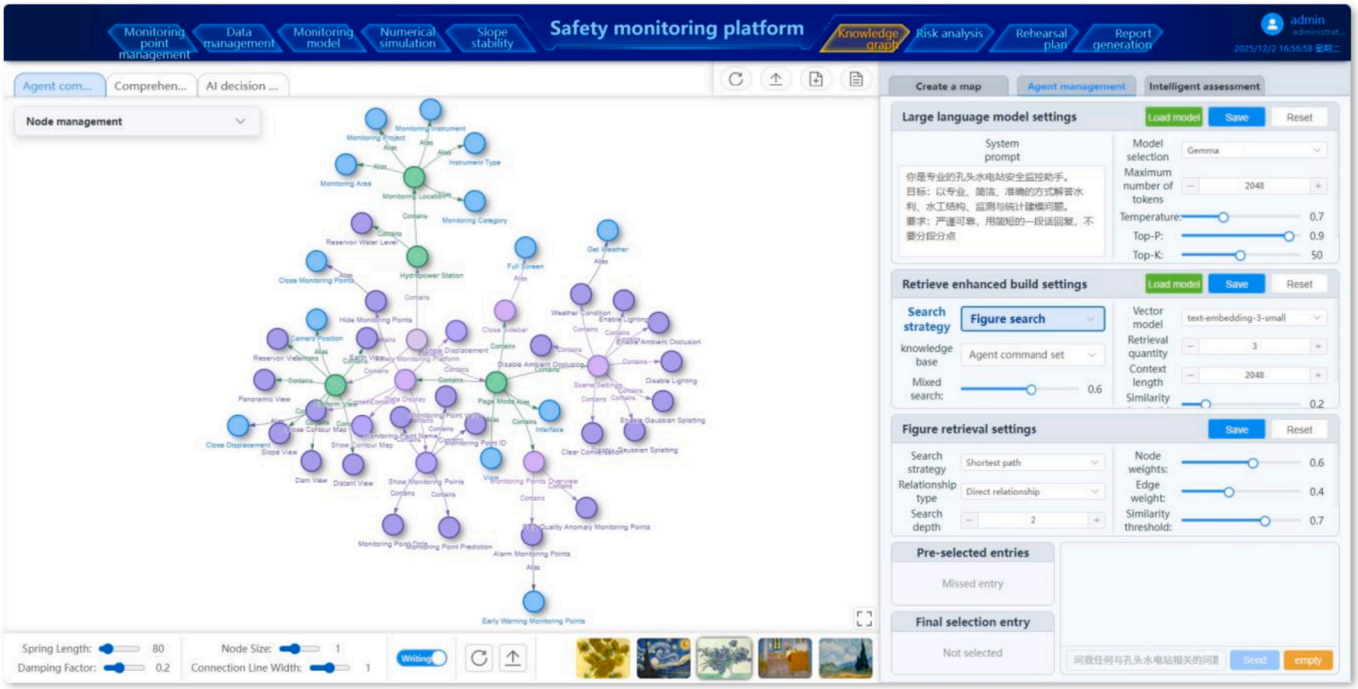


Fig. 15. Interface of the agent assistant management module.

controlling retrieval accuracy and the underlying graph reasoning logic.

Furthermore, similar to the KG management interface, these functional interfaces support the real-time visualization of the current KG structure.

Figs. 16–19 further demonstrate the representative application modes of the proposed agent assistant in the proposed hydropower project safety monitoring DT-based system, including natural language command execution, information retrieval and visualization, and domain-oriented information response.

In Fig. 16, the user issues the natural language command via the dialogue window to hide monitoring points. The agent assistant executes a rapid response, concealing the monitoring points in accordance with the user's instruction. This result demonstrates the assistant's capability for direct natural language-based system operation, thereby reducing the need for conventional menu-dependent manual interaction.

Subsequently, Fig. 17 exhibits the information retrieval and response performance of the agent assistant when handling a query regarding monitoring point information. The agent assistant responds swiftly by visualizing the monitoring point locations within the digital twin engineering scene, while simultaneously displaying data process lines and safety monitoring and risk warning results on the right-side window. Fig. 17. shows that the assistant can connect semantic queries with scene visualization and monitoring analysis results within a unified interaction process.

As illustrated in Fig. 18, the agent assistant displays the slope deformation cloud map through information retrieval and visualization. To ensure smooth responsiveness, medium rendering precision and a resolution scale of 0.6 were enabled. This demonstrates that the assistant can support the visual presentation of engineering analysis outcomes in addition to conventional text-based responses, thereby enhancing the intuitive understanding of monitoring results in the DT environment.

Fig. 19 presents the assistant's domain-oriented information response capability. In response to the user's query regarding the types of slope deformation monitoring instruments, the assistant retrieves relevant domain knowledge and generates an accurate textual answer. This result indicates that the assistant can provide knowledge-grounded consultation services based on the project monitoring context, thereby supporting more efficient human-machine collaboration in hydropower project safety monitoring scenarios.

4.3. Performance evaluation of the lightweight GraphRAG-enabled agent assistant

This section evaluates the proposed agent assistant from three aspects: response performance, lightweight deployment, and user experience. It first constructs a user-instruction dataset, defines the test settings and evaluation metrics, and then compares the standalone Gemma, RAG-enhanced, and GraphRAG-enabled agent assistants. It further analyzes ASR correction performance, interaction efficiency, lightweight KG maintenance, resource consumption, and user satisfaction, thereby verifying the feasibility of the proposed assistant for resource-constrained hydropower project safety monitoring DT systems.

4.3.1. User instruction dataset

In this case study, ten groups of user instructions were selected for testing. These categories are divided into three categories: 3 groups of user instructions about instruction execution, 2 groups of user instructions about information retrieval & visualization, 5 groups of user instructions about Information response. Each group comprises 50 specific queries. Table 6 details the themes and primary contents of each group. The test instructions were sampled from the predefined system-supported instruction space. To ensure evaluation integrity, all system hyperparameters and prompt templates were tuned using a separate development set and fixed prior to the testing phase.

Each group of user instructions mentioned above includes simple instructions, medium-difficulty instructions, and difficult instructions. The difficulty level of the instructions is mainly determined by the length of the agent assistant operation reasoning chain and the size of the retrieved context subgraph, as indicated in Table 7.

4.3.2. Tests and parameter settings

When evaluating the performance of the proposed agent assistant, the Gemma-3-4B was selected as the backbone large language model (LLM). To further optimize memory usage and inference latency, 4-bit



Fig. 16. Agent assistant interface for natural language command execution.

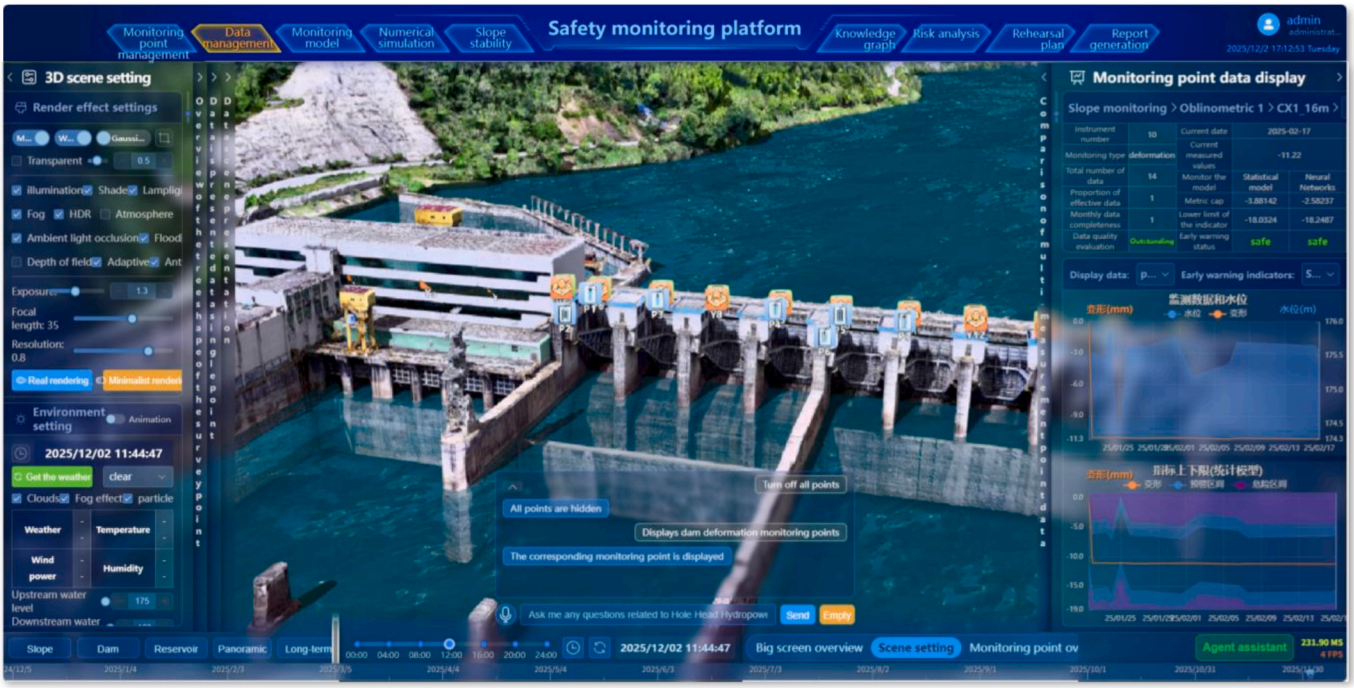


Fig. 17. Agent assistant interface for monitoring-point information retrieval and visualization.



Fig. 18. Agent assistant interface for slope deformation cloud-map retrieval and visualization.

quantization was applied, reducing the model footprint to approximately 3.5GB. The domain Knowledge Graph was hosted on Neo4j. The semantic similarity threshold for conflict resolution was to 0.85. To verify the system's deployment feasibility across different hardware conditions, experiments were conducted in two distinct environments: (1) A high-performance workstation equipped with an Intel Core i5-13600KF CPU, an NVIDIA GeForce RTX 4070 GPU, and 32GB of memory (running CUDA 12.6); (2) A mobile platform with an Intel Core i7-8750H CPU, an NVIDIA GeForce GTX 1060 Max-Q GPU (6GB VRAM), and 16GB of memory (running CUDA 11.6). Both environments operated on Windows 10/11 (64-bit).

The selection of the lightweight Gemma-3-4B model was grounded in the operational characteristics of the hydropower project safety monitoring digital twin system. Unlike open-domain dialogue tasks that require vast parametric knowledge, the proposed agent operates within a bounded functional domain. The system's operational instructions target a finite set of predefined APIs. Empirical studies suggest that for such closed-loop intent classification and slot-filling tasks, the reasoning capability of 4B-parameter models has reached a saturation point sufficient for high-precision execution. By leveraging the GraphRAG

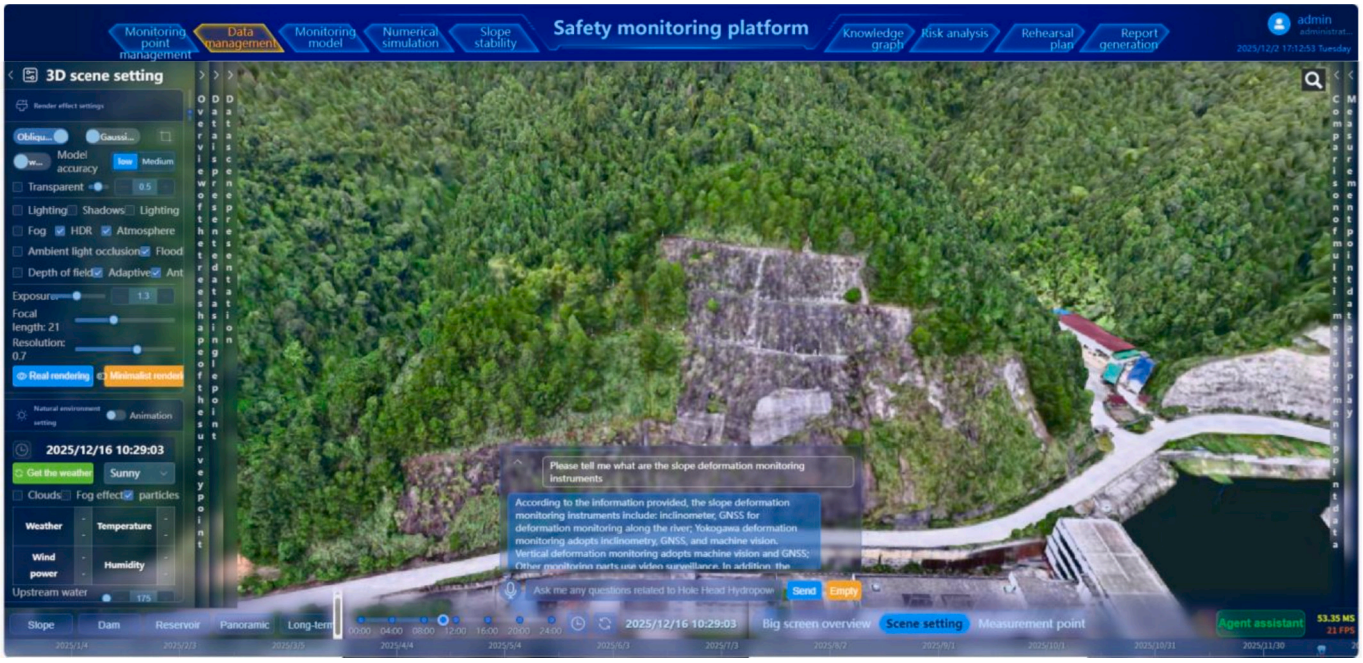


Fig. 19. Agent assistant interface for domain-oriented information response.

framework, complex domain logic and safety constraints are offloaded to the structured KG rather than relying on the LLM's internal weights. The LLM functions primarily as a semantic parser and reasoning engine over the retrieved context, rather than a knowledge repository.

Moreover, the baseline agent assistants based on Gemma-3-4B LLM and standard RAG (representing vector-based retrieval) were utilized as comparisons. This comparative analysis aims to isolate and quantify the specific contributions of the proposed GraphRAG mechanism in enhancing instruction accuracy and safety compliance. Table 8 presents the parameter settings for agent assistant tests based on Gemma-3-4B. To strictly evaluate the contribution of the knowledge graph, we fixed all generation hyperparameters and compared three configurations: the standalone Gemma-3-4B LLM, the agent assistant augmented with RAG, and our proposed lightweight GraphRAG-enabled agent assistant.

To evaluate the lightweight level of the proposed agent assistant, Llama-3-70B as the baseline model was selected to construct the agent assistant as a comparison. To evaluate the efficacy of the lightweight KG maintenance strategy, we generated a synthetic dataset focused on the functional descriptions of the dam safety monitoring digital twin systems. We structured the evaluation into three batches, each containing 20 input items but with shifting distributions of synonym redundancy versus novel entities (ratios of 15:5, 10:10, and 5:15, respectively). This setup specifically tests the system's disambiguation capability under high-interference conditions (Batch 1) and its ingestion efficiency under high-throughput conditions (Batch 3).

4.3.3. Evaluation metrics

To comprehensively evaluate the performance and lightweight level of the proposed agent assistant, we introduce a multi-dimensional evaluation metrics focusing on two core aspects: (1) Performance evaluation metrics, which assesses the accuracy of the answers. (2) Lightweight evaluation metrics, which quantifies the resource overhead and latency reduction achieved by the lightweight incremental update mechanism.

(1) Performance evaluation metrics

1) Accuracy evaluation

To verify the performance of the agent assistant constructed in this system, we selected Precision (P), Recall (R), and F1 score ($F1$) as the key evaluation metrics. The calculation formulas for these metrics are presented in Eqs. (12) ~ (14).

$$P = \frac{TP}{(TP + FP)} \tag{12}$$

$$R = \frac{TP}{(TP + FN)} \tag{13}$$

$$F1 = 2 \times \frac{P \times R}{P + R} \tag{14}$$

where TP (True Positive) denotes the number of users' intents that are correctly identified and executed, FP (False Positive) represents the number of incorrect operations or answers generated by the agent assistant, FN (False Negative) indicates the number of users intents failed to be responded by the agent.

2) Response efficiency

Furthermore, the response efficiency of the agent assistant can be verified by comparing the time T_m, which is required for manual execution of functions, information retrieval, and visualization against the time T_a taken by the agent to respond to user instructions.

(2) Lightweight level evaluation metrics

1) Memory footprint compression

This metric evaluates the resource conservation advantages of the utilized model compared to mainstream LLMs. The indicator is applied for evaluating A higher value indicates that the developed agent assistant is more amenable to deployment within digital twin systems for safety monitoring of small-to-medium hydropower projects, eliminating the reliance on expensive server clusters.

$$R_{mem} = \left(1 - \frac{M_{\text{Gemma}}}{M_{\text{Baseline}}}\right) \times 100\% \tag{15}$$

where M_{Gemma} is the peak VRAM usage of the Gemma-3-4B backbone, M_{Baseline} is the peak VRAM usage of the baseline LLM.

Table 6
Themes and primary contents of user instruction categories.

Catergory ID	Domain	Theme	Primary content
1	Instruction execution	3D view switching	Switching the system focus to specific monitoring zone views, including dam, slope, reservoir, distant, and panoramic views
2	Instruction execution	Rendering & environment settings	Adjusting rendering effects of the digital twin scene, such as enabling/disabling Gaussian Splatting, ambient occlusion, lighting, and weather simulation
3	Information retrieval & visualization	Data visualization control	Managing the visibility of monitoring data layers, including displacement cloud maps, contour maps, and monitoring point markers
4	Instruction execution	Interface interaction management	Adjusting system UI layout, including toggling full-screen mode, sidebar visibility, and clearing conversation history
5	Information retrieval & visualization	Modeling results visualization	Display the analysis and prediction results of various safety monitoring models.
6	Information response	Monitoring system architecture	Inquiring about basic hierarchical information, including monitoring area divisions, project definitions, and instrument classifications
7	Information response	Real-time hydro-meteo & operational data	Current reservoir water levels, dam operational status, and specific environmental parameters
8	Information response	Monitoring point attribute retrieval	Accessing static attribute information of specific monitoring points, such as names, IDs, installation dates, and locations
9	Information response	Alarm & anomaly status inquiry	Listing monitoring points currently in warning or alarm states and querying specific anomalous values and alarm levels
10	Information response	Project basic information retrieval	Retrieving fundamental project archives, covering geographical environment, geological data, and design specifications

2) Entity redundancy rate

This metric assesses the redundancy of newly generated entity nodes within the KG during the ingestion of new documents or Q&A pairs. A high value indicates a prevalence of duplicate nodes, whereas a low value demonstrates the effectiveness of the graph update process. A value closer to 0% indicates superior effectiveness of the conflict resolution strategy, demonstrating the successful elimination of entity redundancy.

$$\mathscr{E}_{red} = \frac{|V_{sys}| - |V_{GT}|}{|V_{GT}|} \times 100\% \tag{16}$$

where $|V_{sys}|$ denotes the total number of entity nodes within the actually generated Knowledge Graph,$|V_{GT}|$ is the number of unique entities in the manually annotated ground truth.

Table 7
Difficulty level of user instructions.

Difficulty level	Length of reasoning chain	Size of retrieved context subgraph	Example
Simple instruction	1–2 hops	Small subgraph (few nodes and edges)	"Switch to the dam perspective"(only on "page mode" and "dam perspective" two nodes and one edge
Medium-difficulty instruction	3–4 hops	Medium-sized subgraph (moderate nodes and edges)	"Tell the user the maximum deformation value for the one specific monitoring point of dam on one specific day." (This involves a subgraph with some nodes and some edges, including nodes for "Dam", "Monitoring point", "Maximum value", and "Deformation value".)
Difficult instruction	5–6 hops	Large subgraph (many nodes and edges)	"Display the piezometer data of all alarm points in the dam area."(This involves a subgraph with multiple nodes and a dozen edges, including nodes for "Dam", "Location", "Instrument Type", "Alarm Monitoring Points", and "Monitoring Point Value". This large subgraph aggregates differenrt information.)

Table 8
Experimental model parameter comparison table.

Type	Temperature	Top-p	Top-k	Retrieval augmentation type
Gemma-3-4B	0.7	0.9	10	None
Gemma-RAG	0.7	0.9	10	Traditional RAG
Gemma-GraphRAG	0.7	0.9	10	GraphRAG

3) Knowledge update speedup

This metric quantifies the speedup ratio of the proposed graph update strategy based on incremental conflict resolution compared to the full reconstruction baseline. A larger value indicates a more significant improvement in the algorithm's graph maintenance efficiency.

$$S_{up} = \frac{T_{Full}}{T_{Inc}} \tag{17}$$

where T_{Full} means the time cost for full graph reconstruction upon new document ingestion, T_{Inc} presents the time cost for the proposed incremental update with conflict resolution.

4.3.4. Response performance evaluation of agent assistants constructed based on three models

(1) To evaluate the effectiveness of agent assistants constructed based on the standalone Gemma-3-4B, the RAG-enabled framework, and the GraphRAG-enabled framework. In this comparative study, all users' instructions were processed as text inputs to eliminate noise interference introduced by Automatic Speech Recognition (ASR) errors, thereby focusing the evaluation specifically on the semantic understanding and intent classification capabilities of the agent assistants. Fig. 20 depicts the normalized confusion matrices illustrating the response accuracy of the agent assistants based on the three models. In these matrices, the vertical axis represents the True

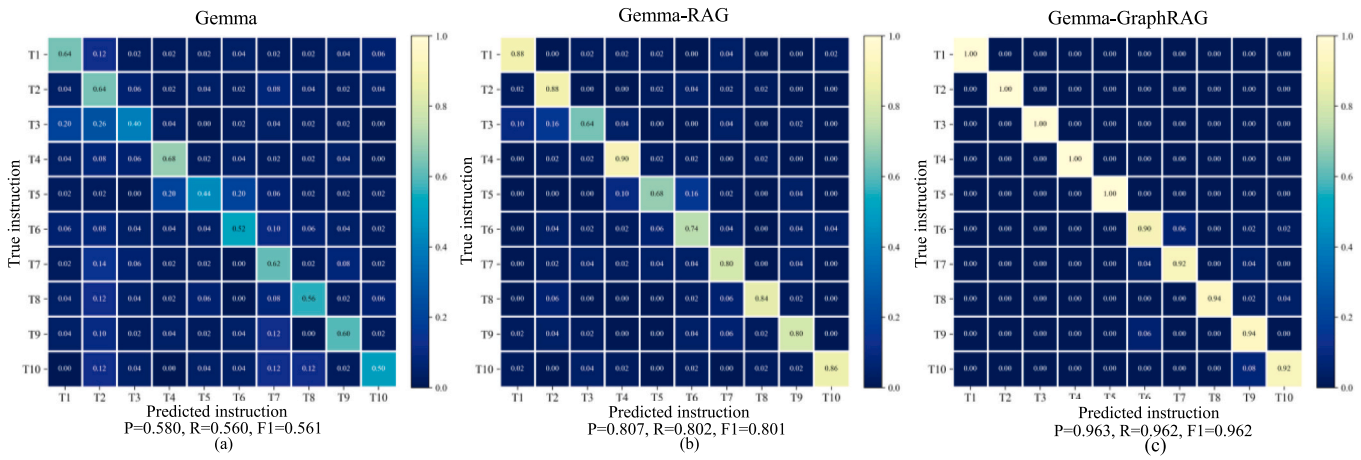


Fig. 20. Performance evaluation results of three agent assistants.

instructions, while the horizontal axis corresponds to the Predicted instructions. The associated performance metrics: Precision (P), Recall (R), and F1 score (*F*1) are detailed at the bottom of each subfigure in Fig. 20.

Fig. 20 presents the confusion matrices for three agent assistants based on three models using a unified test set. The results indicate that the proposed agent assistant, which combines Gemma-based reasoning with GraphRAG-enabled knowledge grounding (Fig. 20(c)), significantly outperforms both the Gemma-based agent assistant (Fig. 20(a)) and the Gemma+RAG-based agent assistant (Fig. 20(b)). The proposed agent assistant achieved an impressive F-score of 0.962.

Specifically, the proposed agent assistant achieved a 100% recall rate (1.00) in user instruction tasks of T1 ~ T5. This demonstrates the strong structural constraints imposed by the knowledge graph. The system functions, such as "View switching" or "Rendering adjustment" are modeled as deterministic nodes with fixed parameters. Consequently, the proposed agent assistant could accurately map natural language commands to unique API trigger paths via entity linking. For operational tasks, the agent works within a closed-set KG environment. This design choice ensures deterministic execution by anchoring generative outputs to fixed system commands, thereby guaranteeing zero-error operations in engineering practical applications. However, for the questions of T6, the performances of each agent assistant were poor. The recall rate for this category is the lowest. We attribute the results to the complexity of understanding the multi-level "Project-Area-Station-Instrument" hierarchy. Unlike single-attribute retrieval, architectural queries often require long-range multi-hop reasoning within the knowledge graph. For instance, identifying "instrument types within a specific area" requires traversing more than one node. Confidence in semantic matching decays as the inference path lengthens. Furthermore, vague user terms regarding "architecture" or "classification" can bias the retrieval scope.

The analysis results of the confusion matrix indicates that errors mainly stem from semantically coupled categories for the agent assistant. For example, T9 (Alarm inquiry) is occasionally misclassified as T7 (Real-time data). Alarms are intrinsically states triggered by real-time data thresholds. Without explicit temporal information or qualifying terms (e.g., asking only about "point status"), the model struggles to distinguish between retrieving raw values and identifying anomalies. Notably, the GraphRAG-enabled agent assistant (Fig. 20(c)) effectively resolves this specific confusion, with T9 showing only minor misclassification into T6 (Predicted instruction).

Moreover, the notably poor performance of both the Gemma-based and Gemma+RAG-based agent assistants on T3 and T5 is attributed to the fact that these categories involve composite tasks of data retrieval coupled with visualization rendering. Unlike general operational commands, these tasks require the agent to simultaneously handle data fetching and graphical parameter configuration. Without structured constraints, standard models struggle to align user intents with the complex API requirements for visualization. In summary, the GraphRAG-enabled agent assistant demonstrates a distinct advantage in handling system operation commands. In contrast, the other two agent assistants frequently misclassify user intents. The reason is that operational commands often share semantic overlap with informational queries in hydropower project safety monitoring systems. Lacking the explicit entity type constraints provided by the knowledge graph, the other two agent assistants are easily misled by the textual ambiguity in user inputs.

(2) Statistical performance comparison

We conducted a statistical analysis based on the performance distribution across the 10 distinct task categories (T1-T10). Rather than relying solely on a single mean score, we calculated the standard deviation of the F1 score across the 10 task groups. As shown in Fig. 21, the proposed Gemma-GraphRAG achieves not only the highest mean score (0.962) but also the lowest standard deviation (±0.041), compared to Gemma-RAG (±0.086) and Gemma (±0.091). This indicates that our method is exceptionally stable across tasks of varying difficulty. The standard deviations reported here reflect the variability across different task categories rather than repeated-run stochastic variance.

Furthermore, we performed a Paired *t*-test on the 10 task groups. The results are summarized in Table 9. The comparison between GraphRAG and RAG yields a *p*-value of <0.001, statistically confirming the significance of the improvement.

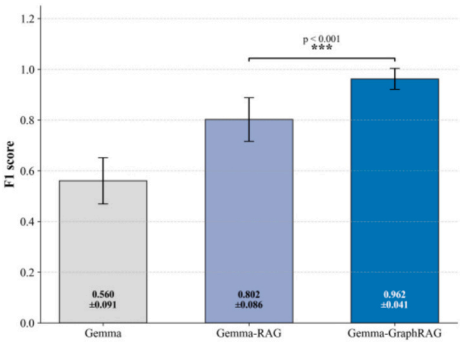


Fig. 21. Statistical performance comparison of different system agent assistants.

Table 9
Statistical significance analysis (paired t-test) across 10 task groups.

Comparison pair	Mean difference	T-statistic	P-value	Significance level
Gemma-RAG vs. Gemma	+0.242	14.85	< 0.001	Significant
Gemma-GraphRAG vs. Gemma-RAG	+0.160	11.42	< 0.001	Significant
Gemma-GraphRAG vs. Gemma	+0.402	16.51	< 0.001	Significant

(3) Before evaluating the practical performance of the graph-constrained ASR error correction method, we calibrated the fusion parameters in Eq. (7) on a small development set by grid search, where the phonetic-weight α, semantic-weight β, and the acceptance threshold τ were selected to maximize the intent-recognition F1 score while controlling false corrections. Fig. 22 presents a user-level calibration analysis for the proposed graph-constrained ASR error correction module. For each parameter setting, we first computed the intent-recognition metrics per user. We selected 10 users, 10 voice commands targeting the aforementioned 10 categories of typical user requirements. The results by the mean performance with inter-user variability is summarized in Fig. 22.

As shown in Fig. 22 (a), the mean F1 score increases with the phonetic weight α from 0.5 to 0.7, indicating that phonetic similarity is highly discriminative for homophone-dominated ASR errors. When α becomes overly large ($\alpha \geq 0.8$), F1 decreases, suggesting reduced robustness under speaker variations where semantic compensation is beneficial. Therefore, we select $\alpha = 0.7$ and set $\beta = 1 - \alpha = 0.3$. Fig. 22 (b) further demonstrates the expected threshold trade-off: increasing the acceptance threshold τ improves precision but reduces recall because more borderline-yet-correct matches are rejected. Consequently, overly stringent threshold leads to a lower F1 despite higher precision. Balancing overall accuracy and practical usability, we choose $\tau = 0.80$ as the operating point. The parameters are then fixed as $\alpha = 0.7$, $\beta = 0.3$, and $\tau = 0.80$ for subsequent tests and validated.

To evaluate the practical performance of the graph-constrained ASR error correction method, the selected 10 users were asked to create a new set of voice commands by re-selecting instructions across the aforementioned 10 categories of typical user requirements, yielding 100 tests in total. For all the tests, the graph-enhanced agent assistant was applied for response, and the response performances of the agent assistant were evaluated under two conditions: with the introduction of the graph-constrained ASR error correction method and without the method. Fig. 23 indicates the response accuracy of the agent assistants without and with ASR correction across 10 requirement categories in the format of radar chart. It compares the Precision, Recall, and F1 scores under two situations. The red solid area significantly encloses the blue dashed area. This demonstrates that the graph-constrained ASR correction method enables agent assistant to achieve better response performance in all categories of user response tasks.

Without the correction mechanism, the average F1 score of the agent assistant hovers between 0.5 and 0.7. This reflects the limitations of general ASR models in processing domain-specific terminology. When questions contain many non-general terms, ASR is prone to homophone substitution errors. This acoustic input distortion prevents the downstream intent classifier from matching keywords. As a result, this leads to a lower recall rate.

With graph-constrained ASR error correction, the response accuracy of agent assistant is secured. However, it exhibits a distinct "right-left asymmetry" feature. This is mainly because system operation requirements (T1 to T5) typically belong to a closed-vocabulary set. Even with accents or noise interference, the graph constraint algorithm only needs to perform fuzzy matching within a limited candidate pool. Thus, it achieves nearly 100% intent restoration. In contrast, the left side of the radar chart (T6 to T10) shows significant improvement, but F1 scores fall to the 0.8–0.9 range. Specifically, T6, T7, and T10 reached only 0.8. This is because these tasks involve open-set named entity recognition, such as specific project or station names. Participant accents may cause pronunciation to deviate significantly from standard Pinyin (exceeding the edit distance threshold). In such cases, multiple phonetically similar candidates may exist in the graph. This leads to higher uncertainty for the correction mechanism. The radar chart results confirm that the graph-constrained ASR error correction method significantly guarantees the response performance of the agent assistant.

(4) To evaluate interaction efficiency, the ten users from the previous tests were asked to manually execute tasks corresponding to each intent through the system human-computer interface. We then compared the average manual execution time of each instruction against the response latency of the agent assistant. Agent response time refers to the system processing latency after input completion. The comparison results are illustrated in Fig. 24.

In Fig. 24, the time of user manual operation and the agent assistant across ten task categories (T1 to T10) is compared. The height of the bars

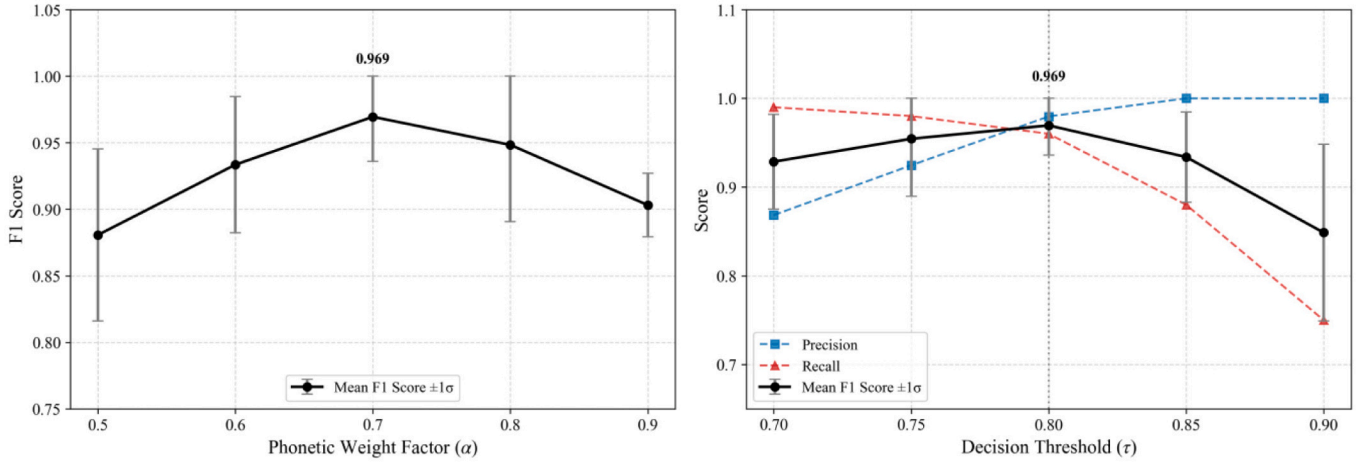


Fig. 22. Mean performance and inter-user variability (±1 standard deviation): (a) under different α and fixed $\tau = 0.8$, (b) under different τ and fixed $\alpha = 0.7$.

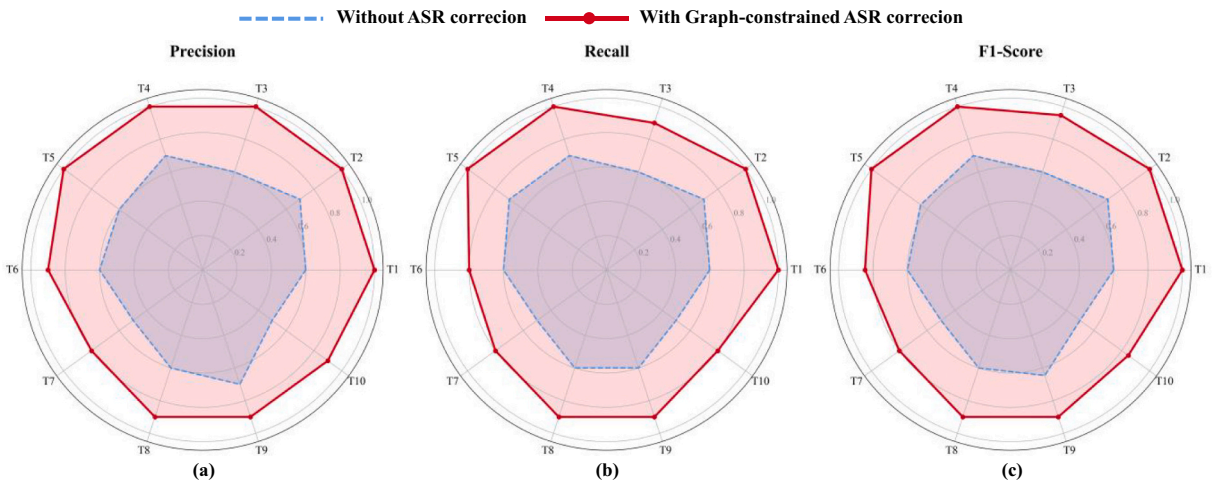


Fig. 23. Radar charts of response accuracy of agent assistant with and without ASR correction.

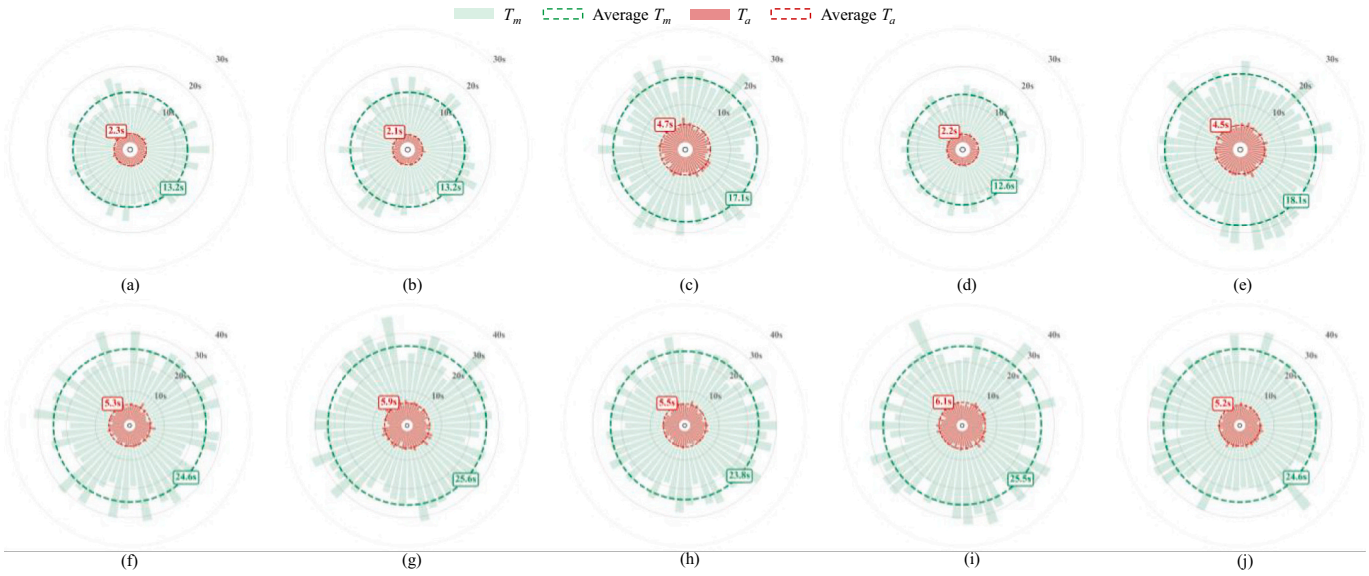


Fig. 24. Comparison results of response time between manual operation and agent assistant.

indicates the average time consumption in seconds. The dashed lines represent the average time consumption of each task category. The overall trend demonstrated that the time consumed by the agent assistant is significantly lower than manual operation. For instruction execution tasks (T1, T2 and T4), the agent assistant reduces the response time to approximately 2.0 s, achieving a 6.5-fold speedup. This result demonstrates the significant gains in operational efficiency achieved by the agent assistant.

This advantage is even more pronounced in complex information retrieval and response tasks (T3, T5, and T6 to T10). The average time for manual retrieval increases to about 25 s, while the agent maintains a response time of around 5 s ~ 6 s. This represents a 4.5-fold increase in efficiency. This reflects the high cognitive load imposed on human users by such tasks. Users must spend significant time searching, filtering, and verifying information. In comparison with manual operation, the response time of agent assistant increases only marginally from around 2 s to 6 s. This demonstrates that the GraphRAG architecture effectively abstracts underlying data complexity during complex information retrieval, improving the system application efficiency.

The green bars in Fig. 24 depict significant variance in manual operations. This indicates differences in operational efficiency among users. Conversely, the negligible error bars for the agent assistant indicate high determinism and robustness. The system consistently delivers rapid responses regardless of task repetition.

4.3.5. *Evaluation of lightweight level of the assistant agents*

The deployment environment for the developed agent assistant is detailed in Table 10. This configuration profile further validates the advantages in resource utilization, demonstrating significant lightweight deployment capabilities. By reducing the dependency on high-performance computing clusters, the system is well-suited for widespread implementation in practical engineering scenarios.

Furthermore, to verify the applicability of the agent assistant in

Table 10
Deploy environment.

Operating system	Windows 10 / 11 (64-bit)
CPU	13th Gen Intel(R) Core(TM) i5-13600KF 3.50 GHz
GPU	NVIDIA GeForce RTX 4070
Hard disk	1 TB NVMe SSD
Memory	32.0 GB
Graphics driver	560.94
CUDA	12.6
cuDNN	9.3.0

small-to-medium hydropower project systems, additional tests were conducted using the configuration demonstrated in Table 11. The results indicate that even under these constrained hardware conditions, the web-based agent assistant maintains smooth and flexible operation when advanced rendering effects are disabled.

(1) In the application of hydropower project digital twin system, the agent assistant is frequently required to interpret complex regulations and generate comprehensive technical answers. The complex information response tasks necessitate process extensive context windows and generating long-form responses. This process maximizes the Key-Value (KV) cache accumulation, thereby inducing the peak VRAM footprint. Therefore, we selected user instructions from categories T6 to T10 and conducted tests on the platform detailed in Table 10 to verify the robustness of the agent assistant's responses. Specifically, while the high-performance setup supports simultaneous high-fidelity rendering and LLM inference, resulting in the peak VRAM usage for each instruction depicted in Fig. 25.

Using Llama-3-70B as the baseline model, we determined that $M_{Baseline} \approx 140$ GB based on standard model specifications. By selecting the maximum peak VRAM usage within each task category, we calculated the minimum memory compression metrics using Eq. (15). The results, presented in Table 12, show that the minimum compression rate reaches 95.00%. This demonstrates that the developed agent assistant minimizes reliance on expensive server clusters.

(2) Twenty text instructions were input for each experimental batch, with ratios of synonymous to novel terms set at 15:5 (Batch 1), 10:10 (Batch 2), and 5:15 (Batch 3), respectively. We analyzed the graph construction redundancy and processing speed under these conditions and compared them against the full graph reconstruction baseline. The results calculated based on Eqs. (16) and (17) are illustrated in Fig. 26.

As depicted in Fig. 26 (a), in the most challenging scenario (Batch 1), where 75% of inputs were conflicting synonyms (15 out of 20), the system exhibited a marginal Entity redundancy rate $\mathscr{E}_{red}$ of 20%. This corresponds to only 1 redundant node generated out of 5 ground-truth additions, demonstrating robustness even under extreme semantic noise.

Notably, in subsequent phases (Batch 2 and 3), the $\mathscr{E}_{red}$ dropped to 0%, proving that as the knowledge graph stabilizes, the conflict resolution mechanism effectively eliminates duplication. Concurrently, in Fig. 26 (b), the update speedup improved from 10× to 31.2×, validating the efficiency of the incremental update strategy as the graph scale grows. It should be noted that unlike massive open-domain graphs, the proposed system functionality graphs are typically characterized by domain-specific sparsity and limited entity scale. Therefore, this test setting accurately reflects real-world reconstruction scenarios, validating that the proposed lightweight KG maintenance method balances high precision with superior maintenance efficiency.

4.3.6. System performance and resource consumption

While Section 4.3.5 focuses on relative lightweight evaluation against baseline settings, Section 4.3.6 reports the practical deployment-time resource consumption profile on a consumer-grade platform. To validate the system's "lightweight" claim and its feasibility for deployment in resource-constrained environments, we profiled the peak resource consumption on our test environment illustrated in Table 11. For the GTX 1060 deployment, we temporarily suspended the advanced visualization module during the computation-intensive text generation phase. This measure significantly alleviates VRAM pressure, ensuring that the full context window and complex reasoning capabilities (Tasks T6-T10) remain operational without memory overflow. This strategy prioritizes reasoning fidelity for complex engineering tasks while optimizing memory usage to fit within consumer-grade hardware constraints. The detailed resource profile is presented in Table 13.

Table 11
Test environment.

Operating system	Windows 10 / 11 (64-bit)
CPU	Intel(R) Core(TM) i7-8750H @ 2.20GHz
GPU	NVIDIA GeForce GTX 1060 with Max-Q Design (6 GB)
Hard disk	128 GB SSD + 1 TB HDD
Memory	16.0 GB
Graphics driver	580.97
CUDA	11.6
cuDNN	8.4.1

As shown in Table 13, the peak Video RAM (VRAM) consumption stabilizes at 4.8 GB. This utilization represents an optimal trade-off: it consumes approximately 80% of the GPU's capacity, leaving a 1.2 GB safety margin for the operating system and display rendering to prevent Out-of-Memory (OOM) errors. This demonstrates that the system can be deployed on standard engineering laptops or edge computing modules without requiring data center-grade hardware.

Furthermore, the average knowledge graph query latency is merely 12 ms, confirming that the retrieval-augmented module introduces negligible overhead to the pipeline. Under the high-precision INT8 setting, the system maintains a stable inference speed of ~16 tokens/s, resulting in an average total response time of 1.5 s. This meets the requirement for fluent real-time interaction in safety monitoring scenarios.

4.3.7. User satisfaction and usability evaluation

Beyond objective metrics, we conducted a pilot user study to assess subjective satisfaction. To ensure a comprehensive evaluation, a total of 20 participants were recruited. The participant composition consisted of 8 professional safety engineers and 12 general users. Regarding the task setting, each participant was instructed to complete a set of predefined, realistic scenarios. Specifically, these tasks were divided into three main categories: instruction execution, information retrieval & visualization, and information response. We utilized a 5-point Likert scale (1 = Poor, 5 = Excellent) to evaluate three dimensions:

(1) Perceived latency: how fast the system feels.
(2) Information accuracy: correctness and relevance of the retrieved information.
(3) Usability: ease of interaction.

The user satisfaction and usability evaluation results are illustrated in Fig. 27.

As presented in Fig. 27, the user study results demonstrate strong performance across all three evaluated dimensions. First, information accuracy emerged as the strongest category, with a median score of approximately 4.78 and a highly concentrated distribution. This result is particularly significant given that a subset of the participants consisted of professional safety engineers. Their consistent high ratings validate that the digital twin agent satisfies stringent professional standards for interpreting complex regulations. Second, regarding perceived latency, the results (Median = 4.67) support our claim regarding the effectiveness of the "lightweight" deployment strategy. Specifically, the agent assistant delivers a responsive user experience without noticeable lag. Finally, in terms of usability, the system achieved a solid mean score of 4.63. Notably, a single data point was recorded at 4.0. This score originated from a general user who initially struggled with the prompt interaction workflow. However, this "floor" of 4.0 confirms that even in worst-case scenarios involving operational unfamiliarity, the system maintains a "Good" baseline of usability and avoids critical interaction barriers.

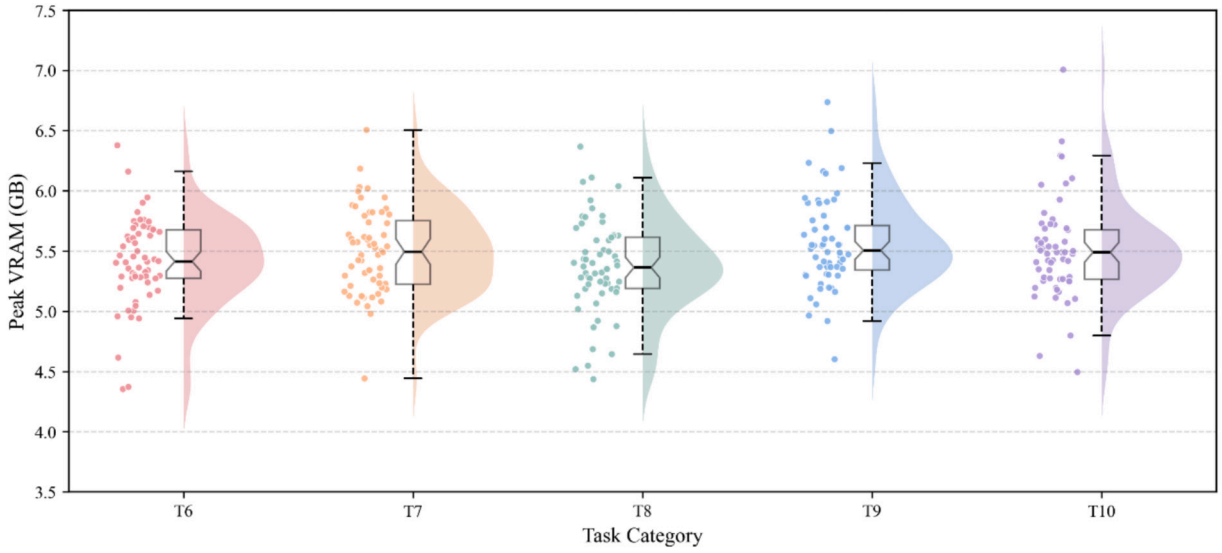


Fig. 25. Values of peak VRAM of users' instructions.

Table 12
Minimum memory footprint compression R_{mem} of different tasks.

Category	T6	T7	T8	T9	T10
R_{mem}	95.43%	95.36%	95.50%	95.14%	95.00%

5. Conclusions and future work

With the aim of addressing key challenges such as the complex system functional execution under resource-constrained deployment and noisy multimodal inputs for hydropower project safety monitoring, this paper presented a lightweight, execution-grounded GraphRAG-enabled multimodal agent assistant. In contrast to existing GraphRAG-based agents that mainly focus on text-only question answering, our contribution tightly integrates task-oriented information retrieval, robust multimodal perception robustness, and lightweight knowledge maintenance for real DT system deployments. Based on the constructed domain KG, a cohesive perception-cognition-execution architecture of agent assistant is proposed. The main conclusion can be drawn as follows:

(1) We developed a hydropower project safety monitoring DT system encompassing core functionalities such as monitoring point management, safety monitoring, and numerical simulation. On this basis, a System-Function-Oriented Knowledge Graph (SFOKG) was constructed. By integrating GraphRAG with the SFOKG and a Gemma-based response module, the agent assistant achieves knowledge-grounded reasoning, task routing, and reliable downstream execution support. As a result, the response accuracy, efficiency, and lightweight deployment of the agent assistant are improved. These characteristics make the proposed framework well suited for small- and medium-scale hydropower projects.

(2) Comparison testing results indicate that the developed agent assistant achieved an overall F1 score of 0.962 across 500 user instruction tests, significantly outperforming the standalone LLM baseline and the standard RAG-augmented baseline. Notably, a 100% recall rate was achieved in system operation tasks.

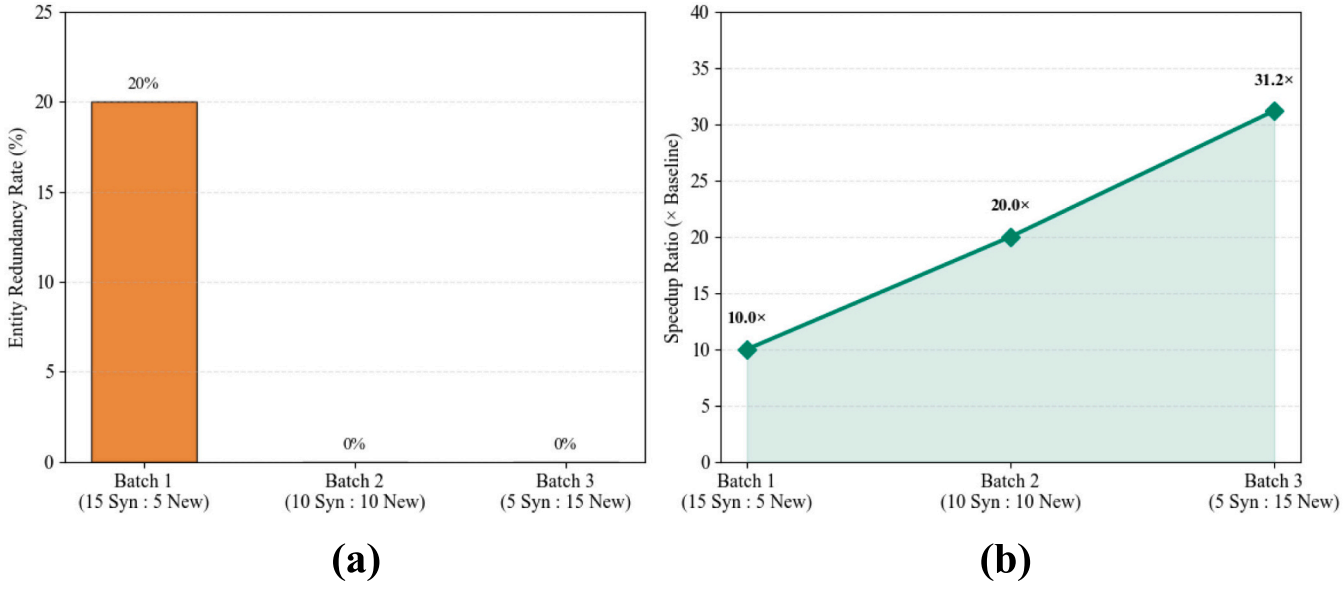


Fig. 26. Lightweight level evaluation metrics calculation results.

Table 13
System resource consumption and performance profile.

Metric	Measured value	Implication
Peak VRAM usage	4.8 GB/6.0 GB	INT8 quantization balances high reasoning fidelity with memory safety (~1.2 GB margin).
Peak system RAM	2.2 GB	Minimal impact on host OS multitasking; KG resides entirely in memory.
Disk storage	~5.0 GB	Includes INT8 model weights, environment dependencies, and the SFOKG (~1.5 MB).
KG query latency	12 ms (Avg.)	Negligible overhead due to the lightweight graph structure (1245 nodes).
LLM inference speed	~16 tokens/s	Sufficient generation speed for fluent human-computer interaction.
Total response time	1.5 s (Avg.)	Provides near-instant feedback for typical engineering queries.

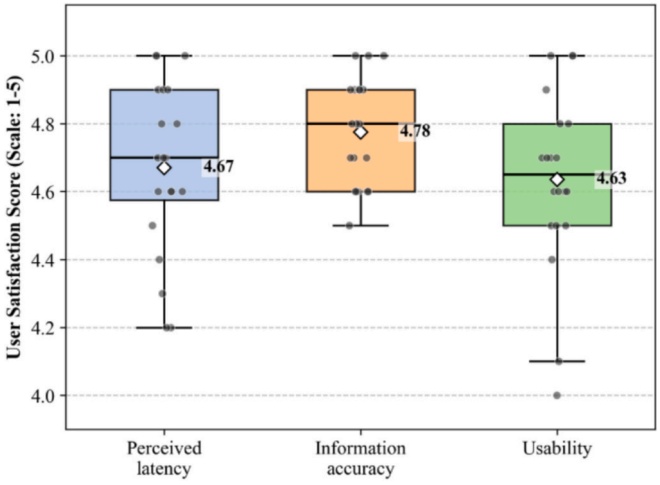


Fig. 27. User satisfaction and usability evaluation.

Furthermore, the graph-constrained ASR error correction module improved the intent recognition F1 score for voice commands from 0.65 (without correction) to 0.91. This result demonstrates exceptional robustness, ensuring accurate intent capture at the perception layer even in acoustically noisy environments. In comparison with manual execution, the agent assistant achieved at least a 4.5-fold speedup in handling the tasks.

(3) In terms of lightweight design, the developed GraphRAG-enabled agent assistant is capable of deployment and operation on a low-cost computing platform, with peak VRAM consumption significantly lower than that of medium and ultra-large-scale LLMs. Furthermore, the proposed conflict resolution-driven incremental KG maintenance strategy enables timely knowledge updates without full graph reconstruction, and provides a feasible way for lightweight applications and low-overhead operation in real engineering systems.

(4) Overall, the proposed agent assistant effectively optimizes interaction workflows and shortens system response time. This study not only technically addresses the interaction bottlenecks of DT systems but also provides a viable pathway for practical application under domain-specific constraints. In particular, it offers a feasible lightweight solution for small- and medium-scale hydropower projects, where deployment portability, maintenance convenience, and user-friendly interaction are of particular importance. However, the current agent assistant primarily targets small- and medium-scale hydropower projects. Future work will extend to more complex scenarios, aiming to achieve stronger generalization in multi-turn interactions, and more comprehensive evaluations. Future work will improve the assistant's generalization capability and accuracy in complex multi-turn interaction scenarios. CRediT authorship contribution statement.

CRediT authorship contribution statement

Chenfei Shao: Visualization, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization, Writing – review & editing, Writing – original draft. **Xiangnan Qin:** Validation, Data curation, Writing – review & editing. **Xing Li:** Validation, Supervision, Funding acquisition. **Meng Yang:** Supervision, Resources. **Dongyang Yuan:** Investigation, Funding acquisition, Writing – review & editing. **Sen Zheng:** Visualization, Validation, Formal analysis, Writing – review & editing, Writing – original draft.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

The research project was supported by the Fundamental Research Funds for the Central Universities (Grant No. B250201005), National Natural Science Foundation of China (Grant No. 52509166), Natural Science Foundation of Henan Province (Grant No. 262300421100), the National Key Research and Development Program of China (Grant No. 2024YFC3210703), China Postdoctoral Science Foundation (Grant No. 2025M773171), the Natural Science Foundation of Jiangxi Province (Grant No. 20252BAC200635), Jiangsu Young Science and Technological Talents Support Project (Grant No. JSTJ-2024-185). The financial support is greatly appreciated by the authors.

Data availability

Data will be made available on request.

References

[1] C. Shao, S. Zheng, Y. Xu, H. Gu, X. Qin, Y. Hu, A visualization system for dam safety monitoring with application of digital twin platform, Expert Syst. Appl. 271 (2025) 126740, https://doi.org/10.1016/j.eswa.2025.126740.
[2] H. Nautiyal, V. Goel, Sustainability assessment of hydropower projects, J. Clean. Prod. 265 (2020) 121661, https://doi.org/10.1016/j.jclepro.2020.121661.
[3] Z. Han, Y. Li, Z. Zhao, B. Zhang, An online safety monitoring system of hydropower station based on expert system, Energy Rep. 8 (2022) 1552–1567, https://doi.org/10.1016/j.egyr.2022.02.040.
[4] M. Liu, S. Fang, H. Dong, C. Xu, Review of digital twin about concepts, technologies, and industrial applications, J. Manuf. Syst. 58 (2021) 346–361, https://doi.org/10.1016/j.jmsy.2020.06.017.
[5] S. Ding, J. Pan, Y. Wang, H. Xu, D.Q. Li, X. Liu, Developing a digital twin for dam safety management, Comput. Geotech. 180 (2025) 107120, https://doi.org/10.1016/j.compgeo.2025.107120.
[6] E. VanDerHorn, S. Mahadevan, Digital twin: generalization, characterization and implementation, Decis. Support. Syst. 145 (2021) 113524, https://doi.org/10.1016/j.dss.2021.113524.
[7] W. Chen, X. Wang, D. Tong, Z. Cai, Y. Zhu, C. Liu, Dynamic early-warning model of dam deformation based on deep learning and fusion of spatiotemporal features, Knowl.-Based Syst. 233 (2021) 107537, https://doi.org/10.1016/j.knosys.2021.107537.
[8] W. Chen, X. Wang, Z. Cai, C. Liu, Y. Zhu, W. Lin, DP-GMM clustering-based ensemble learning prediction methodology for dam deformation considering spatiotemporal differentiation, Knowl.-Based Syst. 222 (2021) 106964, https://doi.org/10.1016/j.knosys.2021.106964.
[9] X. Wu, J. Duan, L. Zhang, S.M. AbouRizk, A hybrid information fusion approach to safety risk perception using sensor data under uncertainty, Stoch. Env. Res. Risk A. 32 (1) (2018) 105–122, https://doi.org/10.1007/s00477-017-1389-9.
[10] C. Shao, S. Zheng, C. Gu, Y. Hu, X. Qin, A novel outlier detection method for monitoring data in dam engineering, Expert Syst. Appl. 193 (2022) 116476, https://doi.org/10.1016/j.eswa.2021.116476.
[11] Z. Rong, R. Pang, B. Xu, Y. Zhou, Dam safety monitoring data anomaly recognition using multiple-point model with local outlier factor, Autom. Constr. 159 (2024) 105290, https://doi.org/10.1016/j.autcon.2024.105290.

[12] J. Tian, R. Yu, J. Chen, C. Chen, Y. Li, X. Sun, H. Huang, Digital twin model for analyzing deformation and seepage in high earth-rock dams, Autom. Constr. 173 (2025) 106079, https://doi.org/10.1016/j.autcon.2025.106079.
[13] J. Tian, C. Chen, L. Zhang, J. Chen, H. Huang, P. Zhang, A digital twin method for real-time analysis of structural deformation and failure for high arch dams, Eng. Struct. 345 (2025) 121426, https://doi.org/10.1016/j.engstruct.2025.121426.
[14] M. Ehteram, H. Karami, S.F. Mousavi, A. El-Shafie, Z. Amini, Optimizing dam and reservoirs operation based model utilizing shark algorithm approach, Knowl.-Based Syst. 122 (2017) 26–38, https://doi.org/10.1016/j.knosys.2017.01.026.
[15] B. Li, J. Yang, D. Hu, Dam monitoring data analysis methods: a literature review, Struct. Control. Health Monit. 27 (3) (2020) e2501, https://doi.org/10.1002/stc.2501.
[16] A. Chen, X. Tang, J. He, New method for measuring diagnosis indexes based on concept lattices in dam health diagnosis, IEEE Access 12 (2024) 22490–22499, https://doi.org/10.1109/ACCESS.2024.3363055.
[17] K. Ma, Z. Yuan, Z. Gao, F. Ren, H. Ke, The early warning method of Dagangshan high-arch dam risk based on the time series prediction of the multivariate monitoring data, Struct. Health Monit. (2024) 14759217241306989, https://doi.org/10.1177/14759217241306989.
[18] I. Zamarrón-Mieza, V. Yepes, J.M. Moreno-Jiménez, A systematic review of application of multi-criteria decision analysis for aging-dam management, J. Clean. Prod. 147 (2017) 217–230, https://doi.org/10.1016/j.jclepro.2017.01.092.
[19] M.F. Allawi, M.L. Ahmed, I.A. Aidan, R.C. Deo, A. El-Shafie, Developing reservoir evaporation predictive model for successful dam management, Stoch. Env. Res. Risk A. 35 (2) (2021) 499–514, https://doi.org/10.1007/s00477-020-01918-6.
[20] M. Raees, I. Meijerink, I. Lykourentzou, V.J. Khan, K. Papangelis, From explainable to interactive AI: a literature review on current trends in human-AI interaction, Int. J. Hum.-Comput. Stud. 189 (2024) 103301, https://doi.org/10.1016/j.ijhcs.2024.103301.
[21] X. Li, S. Wang, S. Zeng, Y. Wu, Y. Yang, A survey on LLM-based multi-agent systems: workflow, infrastructure, and challenges, Vicinagearth 1 (1) (2024) 9, https://doi.org/10.1007/s44336-024-00009-2.
[22] H.C. Jetter, H. Reiterer, F. Geyer, Blended interaction: understanding natural human-computer interaction in post-WIMP interactive spaces, Pers. Ubiquit. Comput. 18 (5) (2014) 1139–1158, https://doi.org/10.1007/s00779-013-0725-4.
[23] Y. Liu, J. Feng, J. Lu, S. Zhou, A review of digital twin capabilities, technologies, and applications based on the maturity model, Adv. Eng. Inform. 60 (2024) 102592, https://doi.org/10.1016/j.aei.2024.102592.
[24] P. Li, X. Hu, B. Xu, X. Zhang, F. Ding, C. Liang, Human-computer interaction design and evaluation of a virtual reality-based comprehensive training system for vascular interventional surgery, Expert Syst. Appl. (2025) 129997, https://doi.org/10.1016/j.eswa.2025.129997.
[25] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W.X. Zhao, Z. Wei, J. Wen, A survey on large language model based autonomous agents, Front. Comp. Sci. 18 (6) (2024) 186345, https://doi.org/10.1007/s11704-024-40231-1.
[26] X. Ge, Z. Chen, L. Yan, Y. Yang, Digital intelligent world: from data-driven AI to knowledge-enabled intelligent agents, IEEE Trans. Knowl. Data Eng. 38 (3) (2026) 1604–1621, https://doi.org/10.1109/TKDE.2026.3651336.
[27] M. Meng, Construction of ecological water resources management leadership model based on multi-agent collaboration, Water Resour. Manag. 38 (15) (2024) 5809–5822, https://doi.org/10.1007/s11269-024-03939-0.
[28] K. Motlaghzadeh, A. Eyni, M. Behboudian, P. Pourmoghim, S. Ashrafi, R. Kerachian, K.W. Hipel, A multi-agent decision-making framework for evaluating water and environmental resources management scenarios under climate change, Sci. Total Environ. 864 (2023) 161060, https://doi.org/10.1016/j.scitotenv.2022.161060.
[29] N. Bahrami, M.R. Nikoo, G. Al-Rawas, K. Al-Jabri, A.H. Gandomi, Optimal treated wastewater allocation among stakeholders based on an agent-based approach, Water Resour. Manag. 37 (1) (2023) 135–156, https://doi.org/10.1007/s11269-022-03359-y.
[30] M.E. Aydin, R. Keleş, A multi agent-based approach for energy efficient water resource management, Comput. Ind. Eng. 151 (2021) 106679, https://doi.org/10.1016/j.cie.2020.106679.
[31] B.S. Ahmed, L.O. Baader, F. Bayram, S. Jagstedt, P. Magnusson, Quality assurance for LLM-RAG systems: Empirical insights from tourism application testing, in: Proceedings of the 2025 IEEE International Conference on Software Testing, Verification and Validation Workshops, Institute of Electrical and Electronics Engineers, 2025, pp. 200–207, https://doi.org/10.48550/arXiv.2502.05782.
[32] P. Laban, A.R. Fabbri, C. Xiong, C.S. Wu, Summary of a haystack: A challenge to long-context LLMs and RAG systems, in: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, Association for Computational Linguistics, 2024, pp. 9885–9903, https://doi.org/10.48550/arXiv.2407.01370.
[33] W. Liao, X. Chu, Y. Wang, TPO: Aligning Large Language Models with Multi-Branch and Multi-Step Preference Trees, arXiv preprint arXiv:2410.12854, 2024, https://doi.org/10.48550/arXiv.2410.12854 (Accessed 6 May 2026).
[34] Y. Feng, Z. Tian, Y. Zhu, Z. Han, H. Luo, G. Zhang, M. Song, CP-Prompt: Composition-based cross-modal prompting for domain-incremental continual learning, in: Proceedings of the 32nd ACM International Conference on Multimedia, Association for Computing Machinery, 2024, pp. 2729–2738, https://doi.org/10.48550/arXiv.2407.21043.
[35] K. Shi, X. Sun, D. Wang, Y. Fu, G. Xu, Q. Li, LLaMA-E: Empowering E-commerce authoring with object-interleaved instruction following, in: Proceedings of the 31st International Conference on Computational Linguistics, Association for Computational Linguistics, Abu Dhabi, United Arab Emirates, 2025, pp. 870–885, https://doi.org/10.48550/arXiv.2308.04913 (Accessed 6 May 2026).
[36] Q. Lin, Y. Zhu, X. Mei, L. Huang, J. Ma, K. He, Z. Peng, E. Cambria, M. Feng, Has multimodal learning delivered universal intelligence in healthcare? A comprehensive survey, Inf. Fusion 116 (2025) 102795, https://doi.org/10.1016/j.inffus.2024.102795.
[37] L.M. Amugongo, P. Mascheroni, S. Brooks, S. Doering, J. Seidel, Retrieval augmented generation for large language models in healthcare: a systematic review, PLOS Digital Health 4 (6) (2025) e0000877, https://doi.org/10.1371/journal.pdig.0000877.
[38] J.A.H. Álvaro, J.G. Barreda, An advanced retrieval-augmented generation system for manufacturing quality control, Adv. Eng. Inform. 64 (2025) 103007, https://doi.org/10.1016/j.aei.2024.103007.
[39] M. Uhm, J. Kim, S. Ahn, H. Jeong, H. Kim, Effectiveness of retrieval augmented generation-based large language models for generating construction safety information, Autom. Constr. 170 (2025) 105926, https://doi.org/10.1016/j.autcon.2024.105926.
[40] G. Papageorgiou, V. Sarlis, M. Maragoudakis, C. Tjortjis, Hybrid multi-agent GraphRAG for E-government: towards a trustworthy AI assistant, Appl. Sci. 15 (11) (2025) 6315, https://doi.org/10.3390/app15116315.
[41] Y. Huang, T.Y. Fung, D.A. DeLaurentis, Addressing complexity in system of systems with GraphRAG: an AI-driven framework for dynamic data integration, Syst. Eng. (2025) e70012, https://doi.org/10.1002/sys.70012.
[42] Y. Chen, G. Lu, K. Wang, S. Chen, C. Duan, Knowledge graph for safety management standards of water conservancy construction engineering, Autom. Constr. 168 (2024) 105873, https://doi.org/10.1016/j.autcon.2024.105873.
[43] Y. Liu, T. Tao, X. Liu, J. Tian, Z. Ren, Y. Wang, X. Wang, Y. Gao, Knowledge graph completion method for hydraulic engineering coupled with spatial transformation and an attention mechanism, Math. Biosci. Eng. 21 (1) (2024) 1394–1412, https://doi.org/10.3934/mbe.2024060.
[44] Q. Wang, Z. Mao, B. Wang, L. Guo, Knowledge graph embedding: a survey of approaches and applications, IEEE Trans. Knowl. Data Eng. 29 (12) (2017) 2724–2743, https://doi.org/10.1109/TKDE.2017.2754499.
[45] C. Zheng, S. Wong, X. Su, Y. Tang, A. Nawaz, M. Kassem, Automating construction contract review using knowledge graph-enhanced large language models, Autom. Constr. 175 (2025) 106179, https://doi.org/10.1016/j.autcon.2025.106179.
[46] F. Ren, L. Zhang, X. Zhao, S. Yin, S. Liu, B. Li, A simple but effective bidirectional framework for relational triple extraction, in: Proceedings of the Fifteenth ACM International Conference on Web Search and Data Mining, Association for Computing Machinery, 2022, pp. 824–832, https://doi.org/10.48550/arXiv.2112.04940.
[47] W. Yan, Y. Shi, Z. Ji, Y. Sui, Z. Tian, W. Wang, Q. Cao, Intelligent predictive maintenance of hydraulic systems based on virtual knowledge graph, Eng. Appl. Artif. Intell. 126 (2023) 106798, https://doi.org/10.1016/j.engappai.2023.106798.
[48] D. Zhang, G. Ma, T. Qu, X. Wang, W. Zhou, X. Wang, A knowledge graph-enhanced large language model for question answering of hydraulic structure safety management, Adv. Eng. Inform. 66 (2025) 103468, https://doi.org/10.1016/j.aei.2025.103468.
[49] Y. Yang, P. Wang, X. Liu, W. Luo, L. Yang, A multi-agent and GraphRAG-based framework for operation and management decision-making in hydraulic projects, Water Resour. Manag. (2025) 1–23, https://doi.org/10.1007/s11269-025-04312-5.
[50] R. Miao, T. Wu, Z. Zhang, Graph RAG-based fault diagnosis for train bogies using knowledge graphs and large language model, Knowl.-Based Syst. 114855 (2025), https://doi.org/10.1016/j.knosys.2025.114855.
[51] Y. Wang, W. Xiong, Y. Zhu, C. Cai, Knowledge graph-driven bridge maintenance decision-making via integrating large language models and chain-of-thought reasoning, Autom. Constr. 181 (2026) 106632, https://doi.org/10.1016/j.autcon.2025.106632.
[52] K. Cao, S. Chen, C. Yang, Z. Li, L. Luo, Z. Ren, Revealing the coupled evolution process of construction risks in mega hydropower engineering through textual semantics, Adv. Eng. Inform. 62 (2024) 102713, https://doi.org/10.1016/j.aei.2024.102713.
[53] A. Kamath, J. Ferret, S. Pathak, N. Vieillard, R. Merhej, S. Perrin, T. Matejovicova, A. Ramé, M. Rivière, L. Rouillard, T. Mesnard, G. Cideron, J.-B. Grill, S. Ramos, E. Yvinec, M. Casbon, E. Pot, I. Penchev, G. Liu, F. Visin, K. Kenealy, L. Beyer, X. Zhai, A. Tsitsulin, R. Busa-Fekete, A. Feng, N. Sachdeva, B. Coleman, Y. Gao, B. Mustafa, I. Barr, E. Parisotto, D. Tian, M. Eyal, C. Cherry, J.-T. Peter, D. Sinopalnikov, S. Bhupatiraju, R. Agarwal, M. Kazemi, D. Malkin, R. Kumar, D. Vilar, I. Brusilovsky, J. Luo, A. Steiner, A. Friesen, A. Sharma, A. Sharma, A. M. Gilady, A. Goedeckemeyer, A. Saade, A. Feng, A. Kolesnikov, A. Bendebury, A. Abdagic, A. Vadi, A. György, A.S. Pinto, A. Das, A. Bapna, A. Miech, A. Yang, A. Paterson, A. Shenoy, A. Chakrabarti, B. Piot, B. Wu, B. Shahriari, B. Petrini, C. Chen, C. Le Lan, C.A. Choquette-Choo, C.J. Carey, C. Brick, D. Deutsch, D. Eisenbud, D. Cattle, D. Cheng, D. Paparas, D.S. Sreepathihalli, D. Reid, D. Tran, D. Zelle, E. Noland, E. Huizenga, E. Kharitonov, F. Liu, G. Amirkhanyan, G. Cameron, H. Hashemi, H. Klimczak-Plucińska, H. Singh, H. Mehta, H.T. Lehri, H. Hazimeh, I. Ballantyne, I. Szpektor, I. Nardini, J. Pouget-Abadie, J. Chan, J. Stanton, J. Wieting, J. Lai, J. Orbay, J. Fernandez, J. Newlan, J.-Y. Ji, J. Singh, K. Black, K. Yu, K. Hui, K. Vodrahalli, K. Greff, L. Qiu, M. Valentine, M. Coelho, M. Ritter, M. Hoffman, M. Watson, M. Chaturvedi, M. Moynihan, M. Ma, N. Babar, N. Noy, N. Byrd, N. Roy, N. Momchev, N. Chauhan, N. Sachdeva, O. Bunyan, P. Botarda, P. Caron, P.K. Rubenstein, P. Culliton, P. Schmid, P.G. Sessa, P. Xu, P. Stanczyk, P. Tafti, R. Shivanna, R. Wu, R. Pan, R. Rokni, R. Willoughby, R. Vallu, R. Mullins, S. Jerome, S. Smoot, S. Girgin, S. Iqbal, S. Reddy, S. Sheth, S. Põder, S. Bhatnagar, S.R. Panyam, S. Eiger, S. Zhang, T. Liu, T. Yacovone, T. Liechty, U. Kalra, U. Evci, V. Misra, V. Roseberry, V. Feinberg, V. Kolesnikov, W. Han,

W. Kwon, X. Chen, Y. Chow, Y. Zhu, Z. Wei, Z. Egyed, V. Cotruta, M. Giang, P. Kirk, A. Rao, K. Black, N. Babar, J. Lo, E. Moreira, L.G. Martins, O. Sanseviero, L. Gonzalez, Z. Gleicher, T. Warkentin, V. Mirrokni, E. Senter, E. Collins, J. Barral, Z. Ghahramani, R. Hadsell, Y. Matias, D. Sculley, S. Petrov, N. Fiedel, N. Shazeer, O. Vinyals, J. Dean, D. Hassabis, K. Kavukcuoglu, C. Farabet, E. Buchatskaya, J.-B. Alayrac, R. Anil, D. Lepikhin, S. Borgeaud, O. Bachem, A. Joulin, A. Andreev, C. Hardin, R. Dadashi, L. Hussenot, Gemma 3 Technical Report, 2025, https://doi.org/10.48550/arXiv.2503.19786 arXiv preprint arXiv:2503.19786, (Accessed 6 May 2026).

[54] M. Abdin, J. Aneja, H. Awadalla, A. Awadallah, A. Awan, N. Bach, A. Bahree, A. Bakhtiari, J. Bao, H. Behl, A. Benhaim, M. Bilenko, J. Bjorck, S. Bubeck, M. Cai, Q. Cai, V. Chaudhary, D. Chen, D. Chen, W. Chen, Y.-C. Chen, Y.-L. Chen, H. Cheng, P. Chopra, X. Dai, M. Dixon, R. Eldan, V. Fragoso, J. Gao, M. Gao, M. Gao, A. Garg, A. Del Giorno, A. Goswami, S. Gunasekar, E. Haider, J. Hao, R. J. Hewett, W. Hu, J. Huynh, D. Iter, S. A. Jacobs, M. Javaheripi, X. Jin, N. Karampatziakis, P. Kauffmann, M. Khademi, D. Kim, Y. J. Kim, L. Kurilenko, J. R. Lee, Y. T. Lee, Y. Li, Y. Li, C. Liang, L. Liden, X. Lin, Z. Lin, C. Liu, L. Liu, M. Liu, W. Liu, X. Liu, C. Luo, P. Madan, A. Mahmoudzadeh, D. Majercak, M. Mazzola, C. C. T. Mendes, A. Mitra, H. Modi, A. Nguyen, B. Norick, B. Patra, D. Perez-Becker, T. Portet, R. Pryzant, H. Qin, M. Radmilac, L. Ren, G. de Rosa, C. Rosset, S. Roy, O. Ruwase, O. Saarikivi, A. Saied, A. Salim, M. Santacroce, S. Shah, N. Shang, H. Sharma, Y. Shen, S. Shukla, X. Song, M. Tanaka, A. Tupini, P. Vaddamanu, C. Wang, G. Wang, L. Wang, S. Wang, X. Wang, Y. Wang, R. Ward, W. Wen, P. Witte, H. Wu, X. Wu, M. Wyatt, B. Xiao, C. Xu, J Xu, W. Xu, J. Xue, S. Yadav, F. Yang, J. Yang, Y. Yang, Z. Yang, D. Yu, L. Yuan, C. Zhang, C. Zhang, J. Zhang, L. L. Zhang, Y. Zhang, Y. Zhang, Y. Zhang, X. Zhou, Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone, 2024, doi:10.48550/arXiv.2404.14219. arXiv preprint arXiv:2404.14219, (Accessed 6 May 2026).

[55] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv, C. Zheng, D. Liu, F. Zhou, F. Huang, F. Hu, H. Ge, H. Wei, H. Lin, J. Tang, J. Yang, J. Tu, J. Zhang, J. Yang, J. Yang, J. Zhou, J. Zhou, J. Lin, K. Dang, K. Bao, K. Yang, L. Yu, L. Deng, M. Li, M. Xue, M. Li, P. Zhang, P. Wang, Q. Zhu, R. Men, R. Gao, S. Liu, S. Luo, T. Li, T. Tang, W. Yin, X. Ren, X. Wang, X. Zhang, X. Ren, Y. Fan, Y. Su, Y. Zhang, Y. Zhang, Y. Wan, Y. Liu, Z. Wang, Z. Cui, Z. Zhang, Z. Zhou, Z. Qiu, Qwen3 Technical Report, 2025, https://doi.org/10.48550/arXiv.2505.09388 arXiv preprint arXiv:2505.09388, (Accessed 6 May 2026).